TALENTLENS — Complete Project Walkthrough

> An enterprise-grade, fully containerized microservices platform that scrapes, cleans, deduplicates, enriches, verifies, and indexes job listings — powered by ML-based salary estimation, scam detection, and semantic vector search.

This consolidated guide walks through every part of the TALENTLENS codebase line-by-line, explains the architecture, and serves as both a learning resource and a reference manual.

Table of Contents

1. High-Level Project Structure
2. The 8-Stage Data Pipeline
3. Line-by-Line: `shared/database/session.py`
4. Line-by-Line: `shared/kafka/producer.py`
5. Line-by-Line: `shared/kafka/consumer.py`
6. Line-by-Line: `services/gateway/main.py`
7. Line-by-Line: `services/collector/main.py`
8. Line-by-Line: `docker-compose.yml`
9. Line-by-Line: `Dockerfile.worker`
10. Line-by-Line: `services/collector/agents.py`
11. Line-by-Line: `services/gateway/routers/auth.py`
12. Line-by-Line: `services/gateway/routers/jobs.py`
13. The 3 ML Models: Salary, Scam, Skills
14. All 5 Worker Services Explained
15. The Universal Worker Pattern
16. Final Cheat Sheet
17. Useful Commands
18. Glossary of Key Concepts

Part 1: High-Level Project Structure

```
Job Scrapping/
├── services/              # All microservices (8 workers + gateway)
│   ├── gateway/           # FastAPI REST API on port 8000
│   │   ├── main.py        # FastAPI app, lifespan, middleware
│   │   ├── deps.py        # JWT helpers, current_user dependency
```

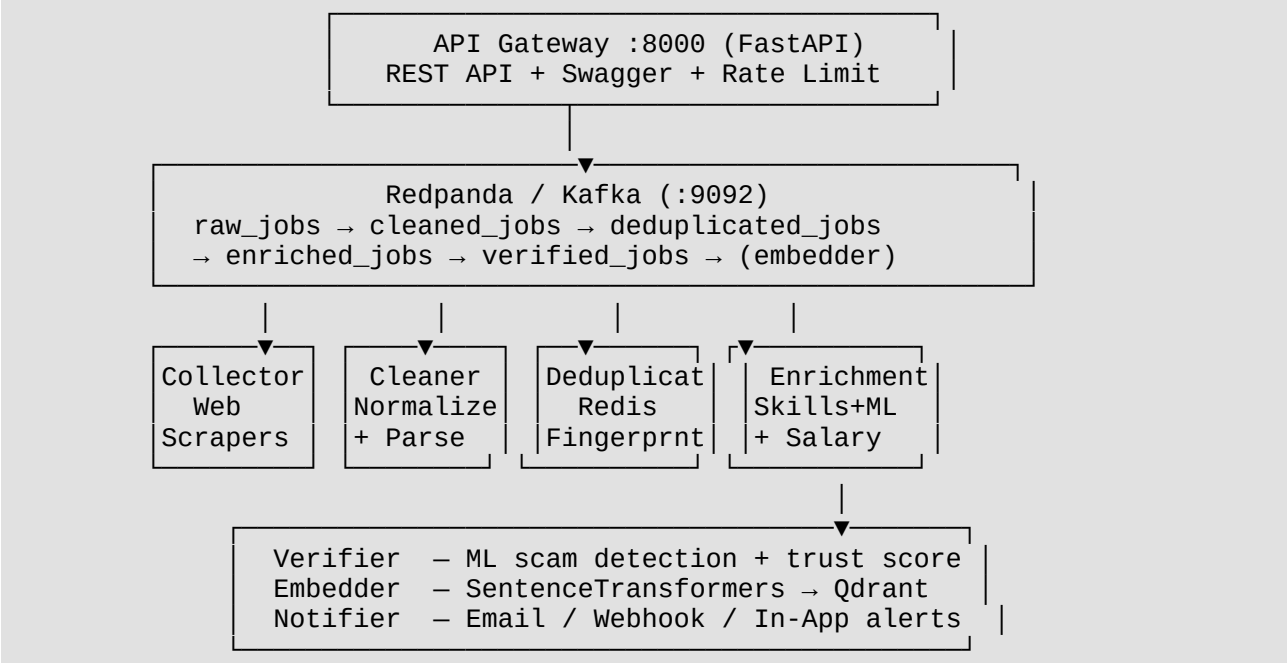
```
│   │   └── routers/       # 8 routers (auth, jobs, search, users, admin,
analytics, notifications, resume)
│   ├── collector/         # Web scraper orchestrator
│   │   ├── main.py        # Kafka producer/consumer + scheduler
│   │   └── agents.py      # 13 collector classes (Adzuna, Greenhouse, Lever,
etc.)
│   ├── cleaner/           # Strips HTML, parses salary
│   ├── deduplicator/      # Removes duplicate jobs via SHA-256 fingerprint
│   ├── enrichment/        # Adds skills + salary estimate
│   ├── verifier/          # ML scam detection + quality score
│   ├── embedder/          # Converts text to vectors for AI search
│   ├── notifier/          # Sends email/webhook alerts
│   ├── search/            # Elasticsearch full-text search
│   ├── analytics/         # Pipeline metrics
│   ├── auth/              # JWT authentication helpers
│   ├── feedback/          # User feedback collection
│   ├── orchestrator/      # Pipeline control
│   └── healer/            # Self-healing watcher
├── shared/                # Reused by every service
│   ├── database/          # MongoDB connection (session.py) + models
(models.py)
│   ├── kafka/             # Async producer/consumer clients + topic constants
│   ├── models/            # Data shapes (Job, User, Company, PipelineEvent)
│   ├── redis/             # Redis client
│   ├── llm/               # OpenAI/Ollama LLM router
│   └── utils/             # Logging, metrics, URL helpers
├── ml/                    # Machine learning models
│   ├── salary_estimator/  # Gradient Boosting model + rule-based fallback
│   ├── scam_detector/     # XGBoost model + 13-rule fallback
│   └── skill_extractor/   # spaCy + regex pipeline
├── frontend/              # UIs
│   ├── jobboard/          # Public site on port 3000 (React + Vite)
│   └── admin/             # Admin dashboard on port 3001
├── infra/                 # Dockerfiles, K8s, observability
│   ├── docker/            # Per-service Dockerfiles
│   ├── k8s/               # Kubernetes manifests
│   └── observability/     # Prometheus + Grafana config
├── tests/                 # Unit + integration tests
├── scripts/               # Helper scripts (init_kafka, health_check)
├── docs/                  # Auto-generated inline docs
├── docker-compose.yml     # Orchestrates all 19 containers
├── docker-compose.prod.yml
├── .env                   # Secrets & config
├── pyproject.toml         # Python dependencies
└── README.md
```

What each top-level folder does

Folder	Purpose
`services/`	13 independent microservices, each runs in its own Docker container. The gateway handles HTTP traffic; the rest are background workers that consume from and produce to Kafka.
`shared/`	Library code imported by every service. Centralizes the MongoDB client, Kafka clients, and data models so behavior is consistent across all workers.
`ml/`	Standalone ML modules with both trained-

Folder	Purpose
	model and rule-based fallbacks. Loaded by the `enrichment`, `verifier`, and `enrichment` workers.
`frontend/`	Two React UIs built with Vite. They never talk to the database directly — only to the gateway's REST API.
`infra/`	Dockerfiles, K8s manifests, and Prometheus/Grafana configuration.
`tests/`	pytest tests. Unit tests don't need infrastructure; integration tests need a running MongoDB.
`scripts/`	One-off utilities: `init_kafka_topics.py`, `health_check.py`, `start_dev.ps1`.
`docs/`	Auto-generated annotated docs for the most important files (used in interview prep).

Part 2: The 8-Stage Data Pipeline



Stage-by-stage breakdown

#	Stage	Input Topic	Output Topic(s)	What it does
1	**Collector**	(web)	`raw_jobs`	Scrapes Adzuna, Indeed, LinkedIn, Naukri, Greenhouse, Lever, Workday,

#	Stage	Input Topic	Output Topic(s)	What it does
				RSS feeds, government portals. Runs every 90 minutes + on demand.
2	**Cleaner**	`raw_jobs`	`cleaned_jobs`	Strips HTML, parses salary strings, normalizes location, extracts job type, generates a content fingerprint.
3	**Deduplicator**	`cleaned_jobs`	`deduplicated_jobs`	Detects near-duplicate jobs using SHA-256 fingerprint of (title, company, location) and (source, source_job_id) pairs.
4	**Enrichment**	`deduplicated_jobs`	`enriched_jobs`	Extracts skills (spaCy NER + regex), estimates salary (LLM + ML rule-based). Adds experience level, domain tags, tech stack.
5	**Verifier**	`enriched_jobs`	`verified_jobs` + `rejected_jobs`	Runs XGBoost scam detection, computes quality score, pings the apply URL, splits jobs into verified vs rejected.
6	**Embedder**	`verified_jobs`	(Qdrant + Mongo)	Converts job text to 384-dim vectors using SentenceTransformers. Upserts to Qdrant for semantic search.
7	**Notifier**	`verified_jobs`	(notifications)	Matches verified jobs against user SavedSearches and logs in-app / email / Telegram /

#	Stage	Input Topic	Output Topic(s)	What it does
				webhook notifications.
8	**Gateway**	(Mongo)	HTTP responses	Serves the entire frontend via REST. Reads from MongoDB, never from Kafka directly.

Key topics created at startup

The `kafka-init` container runs once at boot and creates 24 topics. The worker services declare only the topics they care about. See `scripts/init_kafka_topics.py` for the full list.

Part 3: Line-by-Line: `shared/database/session.py`

Purpose: Centralized async MongoDB connection factory. Every service imports `get_mongo_client()` from this file.

```
1   """Async MongoDB session factory using Motor."""
2   from __future__ import annotations
3   import logging
4   import os
5   from collections.abc import AsyncGenerator
6   from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
7   logger = logging.getLogger(__name__)
8
9   MONGO_URI = os.getenv(
10      "MONGO_URI",
11      "mongodb://admin:admin123@localhost:27017/jobplatform?authSource=admin",
12   )
13   # Read connection string from .env, fall back to local dev default
14
15   _client: AsyncIOMotorClient | None = None
16   # Module-level variable (singleton) — the ONE Mongo client used by whole
app
17
18   def get_mongo_client() -> AsyncIOMotorClient:
19       """Lazy-init a single shared Mongo client."""
20       global _client
21       if _client is None:
22           _client = AsyncIOMotorClient(
23               MONGO_URI,
24               serverSelectionTimeoutMS=5000,   # fail fast if Mongo down
25               connectTimeoutMS=5000,
26               maxPoolSize=50,                  # max 50 parallel connections
27               minPoolSize=5,                   # keep 5 idle for speed
28           )
29       return _client
30       # Singleton pattern: create client only on first call, reuse forever
31
32   async def get_db() -> AsyncGenerator[AsyncIOMotorDatabase, None]:
33       """FastAPI dependency that yields a DB handle per request."""
```

```
34        mongo_client = get_mongo_client()
35        db = mongo_client[db_name]
36        yield db
37
38    async def create_tables() -> None:
39        """Create MongoDB indexes on startup (dev only)."""
40        mongo_client = get_mongo_client()
41        db = mongo_client[db_name]
42        try:
43            from shared.database.base import create_all_indexes
44            results = await create_all_indexes(db)
45            total = sum(results.values())
46            logger.info(f"Database initialized: {total} indexes created")
47        except ImportError:
48            logger.warning("shared.database.base unavailable")
49            await db.jobs.create_index(
50                [("source", 1), ("source_job_id", 1)], unique=True
51            )
52        # Indexes make queries fast; unique=True prevents duplicate jobs at DB
level
```

Key takeaways

- **Singleton pattern**: One Mongo client is shared across the entire process. `get_mongo_client()` checks if it exists; if not, creates it; if yes, reuses it. This avoids opening dozens of TCP connections.
- **Connection pool tuning**: `maxPoolSize=50` allows up to 50 concurrent queries; `minPoolSize=5` keeps 5 connections warm so the first request isn't slow.
- **FastAPI dependency**: `get_db()` uses `yield` because FastAPI's `Depends` system supports generator-based dependencies — it runs the code before `yield` before the request, and the code after `yield` after the request finishes (perfect for resource cleanup).
- **Graceful degradation**: If the fancy `create_all_indexes()` helper is missing, the function falls back to creating a single critical index (the unique compound index on `source + source_job_id`).

Part 4: Line-by-Line: `shared/kafka/producer.py`

Purpose: Wraps `aiokafka.AIOKafkaProducer` so any service can send a single message or a batch of messages with one line.

```
1   from __future__ import annotations
2   import asyncio
3   import json
4   import logging
5   import os
6   from typing import Any, Iterable
7   from aiokafka import AIOKafkaProducer
8   logger = logging.getLogger(__name__)
9
10  KAFKA_BOOTSTRAP = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
11  # Kafka broker address from env
12
13  class KafkaProducerClient:
```

```
14      """Wraps aiokafka producer with JSON serialization + batch send."""
15
16      def __init__(self, client_id: str | None = None) -> None:
17          self._producer: AIOKafkaProducer | None = None
18          self._client_id = client_id or "talentlens-producer"
19
20      async def start(self) -> None:
21          """Connect to Kafka."""
22          self._producer = AIOKafkaProducer(
23              bootstrap_servers=KAFKA_BOOTSTRAP,
24              client_id=self._client_id,
25              value_serializer=lambda v: json.dumps(v, default=str).encode(),
26              acks="all",                  # wait for all replicas
27              enable_idempotence=True,     # no duplicate messages
28              compression_type="lz4",      # compress to save bandwidth
29          )
30          await self._producer.start()
31
32      async def stop(self) -> None:
33          if self._producer:
34              await self._producer.stop()
35
36      async def send(self, topic: str, value: dict, key: str | None = None):
37          """Send a single message."""
38          if not self._producer:
39              raise RuntimeError("Producer not started")
40          k = key.encode() if key else None
41          await self._producer.send_and_wait(topic, value=value, key=k)
42
43      async def send_batch(self, topic: str, messages: Iterable[tuple]):
44          """Send many messages in parallel."""
45          items = list(messages)
46          if not items:
47              return 0
48          sem = asyncio.Semaphore(50)   # max 50 in-flight
49          sent = 0
50          async def _send_one(payload, key):
51              async with sem:
52                  await self.send(topic, payload, key)
53              return 1
54          tasks = [_send_one(p, k) for p, k in items]
55          results = await asyncio.gather(*tasks, return_exceptions=True)
56          sent = sum(1 for r in results if r == 1)
57          return sent
```

Key takeaways

- **`acks="all"`**: Producer waits for ALL replicas to acknowledge before considering a write successful. Safer but slower; used here because we don't want to lose jobs.
- **`enable_idempotence=True`**: Prevents duplicate messages on retry (Kafka may otherwise send a message twice if the network drops between broker write and client ack).
- **`compression_type="lz4"`**: Compresses messages before sending. Job payloads are JSON (text), so compression is very effective (~5-10× smaller).
- **`send_batch()` with semaphore**: Sends N messages concurrently but caps in-flight at 50 so we don't overwhelm the broker. `asyncio.gather(..., return_exceptions=True)` means one failure doesn't kill the whole batch.

- **`value_serializer=lambda v: json.dumps(v, default=str).encode()`**: `default=str` is a trick — if a value is a `datetime` or `ObjectId` that JSON doesn't know how to serialize, it falls back to its `str()` representation.

Part 5: Line-by-Line: `shared/kafka/consumer.py`

Purpose: Wraps `aiokafka.AIOKafkaConsumer` with both single-message and batch-consume modes.

```
1   from __future__ import annotations
2   import asyncio
3   import json
4   import logging
5   import os
6   from typing import Any, Awaitable, Callable
7   from aiokafka import AIOKafkaConsumer
8   logger = logging.getLogger(__name__)
9
10  class KafkaConsumerClient:
11      def __init__(
12          self,
13          topics: list[str],                  # topics to listen on
14          group_id: str,                      # consumer group name
15          max_poll_records: int = 500,
16      ) -> None:
17          self.topics = topics
18          self.group_id = group_id
19          self.max_poll_records = max_poll_records
20          self._consumer: AIOKafkaConsumer | None = None
21
22      async def start(self) -> None:
23          self._consumer = AIOKafkaConsumer(
24              *self.topics,
25              bootstrap_servers=os.getenv("KAFKA_BOOTSTRAP_SERVERS",
"localhost:9092"),
26              group_id=self.group_id,
27              value_deserializer=lambda b: json.loads(b.decode()),
28              enable_auto_commit=False,        # we commit manually after
success
29              max_poll_records=self.max_poll_records,
30          )
31          await self._consumer.start()
32
33      async def stop(self) -> None:
34          if self._consumer:
35              await self._consumer.stop()
36
37      async def consume(self, handler: Callable):
38          """Read one message at a time and call handler(message)."""
39          async for msg in self._consumer:
40              try:
41                  await handler(msg.value)
42                  await self._consumer.commit()
43              except Exception as e:
44                  logger.exception("Handler error: %s", e)
45
46      async def consume_batch(self, handler, batch_size: int = 100,
timeout_ms: int = 1000):
```

```
47          """Read many messages, then call handler(batch)."""
48          while True:
49              batch = await self._consumer.getmany(
50                  timeout_ms=timeout_ms, max_records=batch_size
51              )
52              records = []
53              for tp, msgs in batch.items():
54                  records.extend(msgs)
55              if not records:
56                  continue
57              try:
58                  await handler([m.value for m in records])
59                  await self._consumer.commit()
60              except Exception as e:
61                  logger.exception("Batch handler failed: %s", e)
```

Key takeaways

- **Manual offset commit**: `enable_auto_commit=False` is critical. If we auto-commit and the handler crashes, we lose the message. By committing manually *after* the handler succeeds, we guarantee **at-least-once** delivery (we might process a message twice, but never lose one).
- **`group_id`**: Kafka uses this to load-balance. Two consumers in the same group split the partitions between them. Two consumers in different groups both get every message.
- **`getmany()` for batch mode**: Instead of processing one at a time, we collect up to `batch_size` records (default 100) over `timeout_ms` (default 1s), then call the handler with a list. This is 10-50× faster than per-message for high-throughput workers.
- **Exception swallowing**: If a single message's handler throws, we log it and keep going. The failing message will be re-delivered on next poll (because we never committed its offset), giving us a poor-man's retry loop.

Part 6: Line-by-Line: `services/gateway/main.py`

Purpose: The FastAPI application entrypoint. Wires up all routers, middleware, lifespan, and health checks.

```
1   """FastAPI API Gateway — The single entry point for all external requests.
2   Provides full CRUD APIs, search, auth, and orchestration control.
3   """
4   from __future__ import annotations
5
6   import logging, os, time
7   from contextlib import asynccontextmanager
8
9   from fastapi import FastAPI, Request
10  from fastapi.middleware.cors import CORSMiddleware
11  from fastapi.middleware.gzip import GZipMiddleware
12  from slowapi import Limiter, _rate_limit_exceeded_handler
13  from slowapi.errors import RateLimitExceeded
14  from slowapi.util import get_remote_address
15
16  from services.gateway.routers import auth, jobs, search, users, admin,
analytics, notifications
```

```
from shared.database.session import create_tables
from shared.utils.logging import setup_logging
from shared.utils.metrics import setup_metrics

logger = logging.getLogger(__name__)
limiter = Limiter(key_func=get_remote_address)
# Rate limiter keys by requester IP — 100 req/min per IP by default

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Runs on startup and shutdown."""
    logger.info("Starting Job Intelligence Platform API Gateway")
    if os.getenv("APP_ENV", "development") == "development":
        await create_tables()    # create MongoDB indexes
        logger.info("Database tables initialized")
    yield                         # ← app runs here
    logger.info("Shutting down API Gateway")

app = FastAPI(
    title="Job Intelligence Platform API",
    version="1.0.0",
    lifespan=lifespan,
)

# ── Middleware ──
app.add_middleware(GZipMiddleware, minimum_size=1000)
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.getenv("CORS_ORIGINS",
"http://localhost:3000,http://localhost:3001").split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Request timing ──
@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time-Ms"] = f"{(time.perf_counter() -
start)*1000:.2f}"
    return response

# ── Mount routers ──
app.include_router(auth.router,         prefix="/api/v1/auth",
tags=["Auth"])
app.include_router(jobs.router,         prefix="/api/v1/jobs",
tags=["Jobs"])
app.include_router(search.router,       prefix="/api/v1/search",
tags=["Search"])
app.include_router(users.router,        prefix="/api/v1/users",
tags=["Users"])
app.include_router(admin.router,        prefix="/api/v1/admin",
tags=["Admin"])
app.include_router(analytics.router,    prefix="/api/v1/analytics",
tags=["Analytics"])

app.include_router(notifications.router,prefix="/api/v1/notifications",tags=["No
tifications"])

# ── Health endpoints ──
@app.get("/")
async def root():
```

```
71      return {"name": "Job Intelligence Platform", "version": "1.0.0"}
72
73  @app.get("/health")
74  async def health():
75      return {"status": "healthy"}
76```

### Key takeaways

- **`@asynccontextmanager` + `lifespan`**: Modern FastAPI way to run startup/shutdown logic. The code before `yield` runs at startup; the code after `yield` runs at shutdown.
- **Middleware order matters**: GZip is added before CORS, so responses are gzipped first, then CORS headers are added.
- **`include_router(prefix=...)`**: Every endpoint in `jobs.py` (e.g., `@router.get("")`) is now reachable at `/api/v1/jobs`.
- **`add_process_time_header` middleware**: Adds `X-Process-Time-Ms` to every response — invaluable for debugging slow endpoints.
- **CORS origins from env**: `CORS_ORIGINS` env var is a comma-separated list. In dev, only `localhost:3000` and `localhost:3001` are allowed. In production, set this to your real frontend domain.

---

## Part 7: Line-by-Line: `services/collector/main.py`

**Purpose:** Orchestrates all 13 web scrapers. Runs them on a schedule and on-demand (when a Kafka trigger is received), then bulk-publishes the results to the `raw_jobs` topic.

```python
1   """Collector Service — Orchestrates all job collection activities."""
2   from __future__ import annotations
3
4   import asyncio, os
5   from datetime import datetime
6
7   from shared.kafka.consumer import KafkaConsumerClient
8   from shared.kafka.producer import KafkaProducerClient
9   from shared.kafka.topics import TOPICS
10  from shared.models.job import CollectionSource
11  from shared.utils.logging import setup_logging
12  from services.collector.agents import (
13      COLLECTOR_REGISTRY, get_collector, get_all_sources, DEFAULT_SEARCH_TERMS
14  )
15
16  # Source weights control how many jobs each source gets per run
17  SOURCE_WEIGHTS = {
18      "adzuna": 1.0, "indeed": 0.8, "linkedin": 0.7, "naukri": 0.6,
19      "greenhouse": 0.9, "lever": 0.8, "workday": 0.5,
20  }
21
22  def _scale_limits(total_limit: int, sources: list[str]) -> dict[str, int]:
23      """Distribute the total limit across sources based on weights."""
24      active = {s: SOURCE_WEIGHTS.get(s, 0.5) for s in sources}
25      total = sum(active.values()) or 1
26      return {s: max(5, int(total_limit * w / total)) for s, w in
active.items()}
27
28  class CollectorService:
29      def __init__(self) -> None:
30          self.producer = KafkaProducerClient()
31          self.consumer = KafkaConsumerClient(
```
```

```
32              topics=[TOPICS.COLLECTION_TRIGGER],
33              group_id="collector-service",
34          )
35          self.running = False
36          self._collection_lock = asyncio.Lock()  # only one run at a time
37
38      async def start(self) -> None:
39          await self.producer.start()
40          await self.consumer.start()
41          self.running = True
42          asyncio.create_task(self._scheduled_loop())  # background scheduler
43          await self.consumer.consume(self._handle_trigger)  # block on
trigger
44
45      async def _scheduled_loop(self) -> None:
46          """Run collection every COLLECTION_INTERVAL_MINUTES (default 90)."""
47          interval = int(os.getenv("COLLECTION_INTERVAL_MINUTES", "90")) * 60
48          while self.running:
49              await asyncio.sleep(interval)
50              await self._run_collection(sources=get_all_sources(),
51
limit=int(os.getenv("COLLECTION_RUN_LIMIT","5000")))
52
53      async def _handle_trigger(self, msg: dict) -> None:
54          """React to a manual Kafka trigger."""
55          if self._collection_lock.locked():
56              return  # skip if already running
57          async with self._collection_lock:
58              await self._run_collection(
59                  sources=msg.get("sources", get_all_sources()),
60                  limit=msg.get("limit", 5000),
61              )
62
63      async def _run_collection(self, sources, limit):
64          """Run all collectors in parallel and publish results."""
65          per_source_limits = _scale_limits(limit, sources)
66
67          async def _scrape_one(source_name):
68              c = get_collector(source_name)
69              async with c as collector:           # __aenter__ opens session
70                  return source_name, await
collector.collect(limit=per_source_limits[source_name])
71
72          tasks = [_scrape_one(s) for s in sources]
73          results = await asyncio.gather(*tasks)   # run all in parallel
74
75          for source_name, jobs in results:
76              payloads = [(j.model_dump(mode="json"), str(j.id)) for j in
jobs]
77              if payloads:
78                  await self.producer.send_batch(TOPICS.JOB_RAW, payloads)
79
80  async def main():
81      service = CollectorService()
82      await service.start()
83
84  if __name__ == "__main__":
85      asyncio.run(main())
```

Key takeaways

- **Two triggers, one lock**: Collection can be triggered by (a) the scheduled loop every 90

minutes, or (b) a manual Kafka message. `asyncio.Lock()` ensures both never run at the same time (would cause duplicate jobs in `raw_jobs`).

- **`asyncio.create_task(self._scheduled_loop())`**: Runs the scheduler in the background without blocking the main coroutine. The main coroutine then blocks on `consumer.consume(...)` waiting for manual triggers.
- **`async with c as collector`**: `BaseCollector` defines `__aenter__`/`__aexit__` that open/close the `httpx.AsyncClient` session. So `async with` ensures the session is closed even if `collect()` raises.
- **`_scale_limits`**: Distributes a total target (e.g. 5000 jobs) across sources based on weights. Adzuna gets the full 5000, Indeed gets 80% × 5000 = 4000, etc. Each source gets at least 5 jobs to avoid producing 0-result runs.
- **`send_batch` with `model_dump(mode="json")`**: Pydantic models know how to serialize themselves to JSON. `mode="json"` converts datetimes to ISO strings and UUIDs to plain strings.

Part 8: Line-by-Line: `docker-compose.yml`

Purpose: Orchestrates all 19 containers (10 infrastructure + 5 application + 2 frontend + 2 init). One file, one command: `docker compose up -d`.

Top-level structure

```
1
################################################################################
2   # AI-Powered Job Intelligence Platform — Docker Compose
3   # Single command: docker compose up -d
4   # All services + infrastructure in one file.
5
################################################################################
6
7   # ─── Networks ──────────────────────────────────────────────────────────────
8   networks:
9     platform:
10      driver: bridge
11      ipam:
12        config:
13          - subnet: 172.20.0.0/16
14
15  # ─── Volumes ───────────────────────────────────────────────────────────────
16  volumes:
17    mongodb_data:    # Where Mongo saves its files on your disk
18    redis_data:     # Where Redis saves its cache
19    qdrant_data:    # Where Qdrant saves vectors
20    elasticsearch_data:
21    minio_data:
22    redpanda_data:
23    grafana_data:
24    prometheus_data:
25    prometheus_config:
```

```
26    mlflow_data:
27
28  services:
```

Three layers

Layer 1: Infrastructure (lines ~30–224)

These are the "**databases and tools**" every app needs.

```
35    mongodb:
36      image: mongo:7.0
37      container_name: jip-mongodb
38      restart: unless-stopped
39      environment:
40        MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-admin}
41        MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:-admin123}
42        MONGO_INITDB_DATABASE: ${MONGO_DB:-jobplatform}
43      volumes:
44        - mongodb_data:/data/db
45      ports:
46        - "27017:27017"
47      networks:
48        - platform
49      healthcheck:
50        test: ["CMD-SHELL", "echo 'db.runCommand(\"ping\").ok' | mongosh --
quiet || exit 0"]
51        interval: 10s
52        timeout: 5s
53        retries: 5

55    redis:
56      image: redis:7-alpine
57      container_name: jip-redis
58      restart: unless-stopped
59      command: ["sh", "-c", "redis-server --maxmemory 512mb --maxmemory-policy
allkeys-lru --appendonly yes ${REDIS_PASSWORD:+--requirepass $REDIS_PASSWORD}"]
60      volumes:
61        - redis_data:/data
62      ports:
63        - "6379:6379"

77    redpanda:
78      image: docker.redpanda.com/redpandadata/redpanda:latest
79      container_name: jip-redpanda
80      restart: unless-stopped
81      command:
82        - redpanda
83        - start
84        - --smp=1
85        - --memory=512M
86        - --overprovisioned
87        - --node-id=0
88        - --kafka-addr=PLAINTEXT://0.0.0.0:29092,OUTSIDE://0.0.0.0:9092
89        -
--advertise-kafka-addr=PLAINTEXT://redpanda:29092,OUTSIDE://localhost:9092

130   kafka-init:
131     image: docker.redpanda.com/redpandadata/redpanda:v24.3.1
132     container_name: jip-kafka-init
133     restart: "no"               # never restart, exits after creating
topics
```

```
134     networks: - platform
135     depends_on: - redpanda
136     entrypoint: >
137       /bin/sh -c "
138       rpk topic create raw_jobs cleaned_jobs ... --brokers redpanda:29092 ||
true
139       "
```

Layer 2: Application services (lines ~230–432)

```
230   gateway:
231     build:
232       context: .
233       dockerfile: infra/docker/gateway/Dockerfile
234     container_name: jip-gateway
235     restart: unless-stopped
236     environment:
237       - APP_ENV=development
238       - MONGO_URI=mongodb://${MONGO_USER:-admin}:${MONGO_PASSWORD:-admin123}
@jip-mongodb:27017/...
239       - REDIS_URL=redis://redis:6379/0
240       - KAFKA_BOOTSTRAP_SERVERS=redpanda:29092
241       - QDRANT_HOST=qdrant
242       - ELASTICSEARCH_URL=http://elasticsearch:9200
243       - MINIO_ENDPOINT=minio:9000
244       - CORS_ORIGINS=http://localhost:3000,http://localhost:3001
245       - SERVICE_NAME=gateway
246     env_file: - .env
247     ports: - "8000:8000"
248     networks: - platform
249     depends_on: mongodb, redis, redpanda, kafka-init
250     healthcheck:
251       test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
252       interval: 30s
253       timeout: 10s
254       retries: 3
```

Layer 3: Frontends (lines ~438+)

```
438   jobboard:
439     build:
440       context: ./frontend/jobboard
441       dockerfile: Dockerfile
442       target: development
443     container_name: jip-jobboard
444     restart: unless-stopped
445     environment:
446       - VITE_API_URL=http://localhost:8000/api/v1
447     ports: - "3000:3000"
```

Key takeaways

- **Service name = DNS name**: Inside the Docker network, `mongodb`, `redis`, `redpanda`, `qdrant` resolve to the right container. The gateway's `MONGO_URI` references `jip-mongodb` by container name, not `localhost`.
- **`depends_on`**: `kafka-init` runs once after `redpanda` and creates all 24 topics. `gateway` waits for `kafka-init` to finish so topics exist when it starts producing.
- **`healthcheck`**: Docker uses this to know if the container is "healthy". Other services'

`depends_on` blocks until the dependency is healthy.

- **`volumes:`** are persistent — data survives `docker compose down`. Only `docker compose down -v` wipes them.
- **Multi-stage builds**: The frontend uses `target: development` to use a dev server (with hot reload) instead of a static build.

Part 9: Line-by-Line: `Dockerfile.worker`

Purpose: Template Dockerfile used by `cleaner`, `deduplicator`, `enrichment`, `verifier`, and `embedder`. Built per-service via the `SERVICE_NAME` ARG.

```
1   # ── Shared Worker Dockerfile (cleaner, deduplicator, enrichment, verifier,
embedder)
2   FROM python:3.11-slim
3
4   ARG SERVICE_NAME=cleaner
5
6   RUN groupadd --gid 1000 app && useradd --uid 1000 --gid app --shell
/bin/bash --create-home app
7
8   WORKDIR /app
9
10  RUN apt-get update && apt-get install -y --no-install-recommends curl && rm
-rf /var/lib/apt/lists/*
11
12  COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
13
14  COPY pyproject.toml ./
15  COPY shared/ ./shared/
16  COPY services/${SERVICE_NAME}/ ./services/${SERVICE_NAME}/
17
18  # Install base dependencies common to all workers
19  RUN uv pip install --system --no-cache \
20      aiokafka \
21      lz4 \
22      motor \
23      pydantic \
24      httpx
25
26  # Install service-specific extras
27  RUN if [ "$SERVICE_NAME" = "enrichment" ] || [ "$SERVICE_NAME" =
"verifier" ]; then \
28        uv pip install --system --no-cache langchain-core langchain-ollama
langchain-openai langchain-anthropic; \
29      fi && \
30      if [ "$SERVICE_NAME" = "embedder" ]; then \
31        uv pip install --system --no-cache sentence-transformers qdrant-
client; \
32      fi
33
34  USER app
35  ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app
SERVICE_NAME=${SERVICE_NAME}
36
37  CMD ["sh", "-c", "python -m services.${SERVICE_NAME}.main"]
```

Key takeaways

- **`ARG SERVICE_NAME=cleaner`**: Build-time variable. Each service's `docker-compose.yml` passes `--build-arg SERVICE_NAME=enrichment` etc.
- **Non-root user**: `useradd ... app` and `USER app` means the container runs as a non-privileged user. Security best practice.
- **Multi-stage copy for uv**: `COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv` extracts the uv binary from a pre-built image — much faster than installing it via pip.
- **Conditional dependencies**: The `RUN if ... fi` block installs `langchain-*` only for `enrichment` and `verifier`, and `sentence-transformers + qdrant-client` only for `embedder`. This keeps the image small for workers that don't need ML.
- **`PYTHONUNBUFFERED=1`**: Python doesn't buffer stdout when this is set, so logs appear in `docker logs` immediately.
- **`PYTHONPATH=/app`**: So `import services.cleaner.main` works without changing directory.
- **`CMD` with shell expansion**: `python -m services.${SERVICE_NAME}.main` becomes `python -m services.enrichment.main` at container start. This single Dockerfile serves all 5 workers.

Part 10: Line-by-Line: `services/collector/agents.py`

Purpose: All 13 web scrapers + shared infrastructure (CircuitBreaker, retry helper, BaseCollector). About 1,534 lines.

10.1 Constants & search terms (lines 1–102)

```
1   """
2   Job Collection Agents — Multi-source job aggregation.
3   Sources: Adzuna API, Greenhouse, Lever, Workday, Naukri, Indeed, LinkedIn,
4            Company Careers RSS, Remotive, WeWorkRemotely, DailyRemote,
AngelList,
5            TimesJobs, Glassdoor RSS, Government portals.
6   """
7
16  from __future__ import annotations
17
18  import asyncio, hashlib, logging, os, random, re, time
19  from collections import deque
20  from dataclasses import dataclass, field
21  from datetime import datetime
22  from uuid import uuid4
23
24  import httpx
25  from bs4 import BeautifulSoup
26
27  from shared.models.job import CollectionSource, RawJob
28
```

```
29  # 60+ default search terms for the Indian market
30  DEFAULT_SEARCH_TERMS = [
31      "software engineer", "python developer", "data scientist",
32      "machine learning engineer", "devops engineer", ...
33      "bangalore jobs", "pune jobs", "hyderabad jobs",
34  ]
35
36  # 3 fake browser User-Agents, rotated to avoid detection
37  BROWSER_HEADERS_POOL = [
38      {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Chrome/122.0.0.0"},
39      {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ...
Safari/605.1.15"},
40      {"User-Agent": "Mozilla/5.0 (X11; Linux x86_64) ... Chrome/121.0.0.0"},
41  ]
```

10.2 The Circuit Breaker (lines 104–151)

```
104  @dataclass
105  class CircuitBreaker:
106      """Stop sending requests to a failing source for a cool-down period."""
107      failure_threshold: int = 5
108      recovery_timeout_seconds: int = 60
109      _failures: int = 0
110      _open_since: float | None = None
111
112      @property
113      def state(self) -> str:
114          if self._open_since is not None:
115              if (time.monotonic() - self._open_since) >=
self.recovery_timeout_seconds:
116                  return "half_open"   # Time to retry
117              return "open"             # Stop, still broken
118          return "closed"               # All good
119
120      def record_success(self):
121          if self.state == "half_open":
122              self._failures = 0
123              self._open_since = None
124
125      def record_failure(self):
126          self._failures += 1
127          if self._failures >= self.failure_threshold and self._open_since is
None:
128              self._open_since = time.monotonic()
129              # Open the circuit — stop trying for recovery_timeout_seconds
130
131      def allow_request(self) -> bool:
132          return self.state != "open"
```

10.3 The retry helper (lines 156–215)

```
156  async def retry_with_backoff(
157      coro_fn, max_retries: int = 5, initial_delay: float = 1.0,
158      max_delay: float = 60.0, backoff_factor: float = 2.0,
159      retry_on = (httpx.HTTPStatusError, httpx.ConnectError, ...),
160      circuit_breaker: CircuitBreaker | None = None,
161      status_codes_to_retry: set = {429, 500, 502, 503, 504, 408},
162  ):
163      delay = initial_delay
164      for attempt in range(max_retries):
165          if circuit_breaker and not circuit_breaker.allow_request():
```

```
166             raise RuntimeError("Circuit breaker is OPEN")
167         try:
168             result = await coro_fn()
169             if circuit_breaker: circuit_breaker.record_success()
170             return result
171         except httpx.HTTPStatusError as e:
172             if e.response.status_code not in status_codes_to_retry:
173                 raise  # 404, 401 etc. — don't retry
174             if e.response.status_code == 429:
175                 retry_after = e.response.headers.get("Retry-After")
176                 if retry_after and retry_after.isdigit():
177                     delay = max(delay, float(retry_after))
178         except retry_on:
179             pass  # network error — will retry below
180 
181         if circuit_breaker: circuit_breaker.record_failure()
182         jitter = random.uniform(0, delay * 0.5)
183         await asyncio.sleep(min(delay + jitter, max_delay))
184         delay *= backoff_factor
185 
186     raise RuntimeError(f"Failed after {max_retries} retries")
```

10.4 The BaseCollector (lines 218–370)

```
218 class BaseCollector:
219     """Parent class for all 13 scrapers. Provides shared logic."""
220 
221     source: CollectionSource = CollectionSource.UNKNOWN
222 
223     def __init__(self, rate_limit_per_second=None,
max_concurrent_requests=None):
224         self.session: httpx.AsyncClient | None = None
225         self._semaphore: asyncio.Semaphore | None = None
226         self.circuit_breaker = CircuitBreaker(
227             failure_threshold=int(os.getenv("CB_FAILURE_THRESHOLD", "8")),
228             recovery_timeout_seconds=int(os.getenv("CB_RECOVERY_SECONDS",
"90")),
229         )
230         self._rate_multiplier = 1.0  # adaptive — slows on 429, speeds on
200 
231 
232     async def __aenter__(self):
233         self._semaphore = asyncio.Semaphore(self._max_concurrent)
234         self.session = httpx.AsyncClient(
235             headers=random.choice(BROWSER_HEADERS_POOL),
236             timeout=httpx.Timeout(30.0, connect=10.0),
237             follow_redirects=True,
238         )
239         return self
240 
241     async def __aexit__(self, *args):
242         if self.session: await self.session.aclose()
243 
244     async def _rate_limited_get(self, url, **kwargs):
245         if not self.circuit_breaker.allow_request():
246             raise RuntimeError(f"Circuit breaker open for {self.source}")
247         self._rotate_headers()
248         async with self._semaphore:                # concurrency cap
249             effective_rate = max(self.effective_rate, 0.1)
250             min_interval = 1.0 / effective_rate
251             now = time.monotonic()
252             wait = (self._last_request_time + min_interval) - now
253             if wait > 0:
```

```
254                     await asyncio.sleep(wait + random.uniform(0,
min_interval*0.1))
255                 self._last_request_time = time.monotonic()
256                 response = await self.session.get(url, **kwargs)
257
258                 # ── Adaptive rate adjustment ──
259                 if response.status_code in (429, 503):
260                     self._rate_multiplier = max(0.1, self._rate_multiplier *
0.5)  # slow down 50%
261                 elif response.status_code < 400 and (time.monotonic() - now) <
1.0:
262                     self._rate_multiplier = min(2.0, self._rate_multiplier *
1.02)  # speed up 2%
263
264                 if response.status_code >= 500:
265                     self.circuit_breaker.record_failure()
266                 else:
267                     self.circuit_breaker.record_success((time.monotonic() -
now) * 1000)
268
269                 response.raise_for_status()
270                 return response
271
272     async def collect(self, search_terms=None, location=None, limit=100):
273         raise NotImplementedError  # subclasses must implement
274
275     def _generate_fingerprint(self, *parts: str) -> str:
276         content = "|".join(p.lower().strip() for p in parts if p)
277         return hashlib.md5(content.encode()).hexdigest()
```

10.5 The 13 collector classes

All 13 follow the same pattern: subclass `BaseCollector`, set `source = CollectionSource.XXX`, implement `async def collect(...)`.

```
375  class AdzunaCollector(BaseCollector):
376      """Adzuna Jobs API — concurrent per search term."""
377      source = CollectionSource.ADZUNA
378      BASE_URL = "https://api.adzuna.com/v1/api/jobs"
379
380      def __init__(self):
381          super().__init__(
382              rate_limit_per_second=float(os.getenv("ADZUNA_RATE", "4")),
383              max_concurrent_requests=int(os.getenv("ADZUNA_CONCURRENT",
"10")),
384          )
385          self.app_id = os.getenv("ADZUNA_API_ID", "")
386          self.app_key = os.getenv("ADZUNA_API_KEY", "")
387          self.country = os.getenv("ADZUNA_COUNTRY", "in")
388
389      async def collect(self, search_terms=None, location=None, limit=100):
390          if not self.is_configured:
391              return []
392          terms = search_terms or DEFAULT_SEARCH_TERMS
393          sem = asyncio.Semaphore(int(os.getenv("ADZUNA_TERM_PARALLEL",
"5")))
394
395          async def _collect_one(term):
396              async with sem:
397                  try:
398                      return await retry_with_backoff(
399                          lambda: self._collect_term(term, location,
```

```
run_id, ...),
400                             max_retries=4,
circuit_breaker=self.circuit_breaker,
401                         )
402                     except Exception:
403                         return []
404
405             results = await asyncio.gather(*[_collect_one(t) for t in terms])
406             # flatten + dedup + return
```

The 13 collectors are: `Adzuna`, `Greenhouse`, `Lever`, `Workday`, `IndeedScraper`, `NaukriScraper`, `LinkedIn`, `RSSFeed`, `GovernmentJobs`, `CompanyCareers`, `Remotive`, `Arbeitnow`, `Manual`.

10.6 The collector registry (lines 1502–1534)

```
1504 COLLECTOR_REGISTRY: dict[str, type[BaseCollector]] = {
1505     CollectionSource.ADZUNA: AdzunaCollector,
1506     CollectionSource.GREENHOUSE: GreenhouseCollector,
1507     CollectionSource.LEVER: LeverCollector,
1508     CollectionSource.WORKDAY: WorkdayCollector,
1509     CollectionSource.INDEED: IndeedScraper,
1510     CollectionSource.NAUKRI: NaukriScraper,
1511     CollectionSource.LINKEDIN: LinkedInCollector,
1512     CollectionSource.RSS: RSSFeedCollector,
1513     CollectionSource.GOVERNMENT: GovernmentJobsCollector,
1514     CollectionSource.COMPANY_CAREERS: CompanyCareersCollector,
1515     CollectionSource.REMOTIVE: RemotiveCollector,
1516     CollectionSource.ARBEITNOW: ArbeitnowCollector,
1517     CollectionSource.MANUAL: ManualCollector,
1518 }
1519
1521 def get_collector(source: str) -> BaseCollector:
1521     cls = COLLECTOR_REGISTRY.get(source)
1522     if not cls: raise ValueError(f"Unknown source: {source}")
1523     return cls()
1524
1530 def get_all_sources(exclude_manual: bool = True) -> list[str]:
1531     sources = list(COLLECTOR_REGISTRY.keys())
1532     if exclude_manual:
1533         sources = [s for s in sources if s != CollectionSource.MANUAL]
1534     return sources
```

Key takeaways

- **Circuit Breaker pattern**: Like an electrical fuse. After 5 failures, stop trying for 60s. Then "half-open" — let ONE request through to test.
- **Adaptive rate limiting**: Multiplier starts at 1.0. On 429, halves to 0.5 (slow down). On 200, multiplies by 1.02 (gradually speed up to 2.0 max). This automatically finds the right speed for each site.
- **Exponential backoff with jitter**: 1s → 2s → 4s → 8s. The `random.uniform(0, delay * 0.5)` jitter prevents many parallel scrapers from all retrying at the exact same instant.
- **Registry pattern**: `COLLECTOR_REGISTRY` is a name → class map. `get_collector("adzuna")` returns a fresh `AdzunaCollector()` instance. The

`CollectorService` only knows about the registry, not about specific collector classes.

- **`async with c as collector`**: The `__aenter__` opens the HTTP session; `__aexit__` closes it. So even if `collect()` raises, the session is cleaned up.

Part 11: Line-by-Line: `services/gateway/routers/auth.py`

Purpose: JWT-based authentication: register, login, refresh, logout, me, change-password, and one-time admin bootstrap.

11.1 Imports and setup (lines 1–35)

```
1   """
2   Auth router — JWT-based authentication: register, login, refresh, logout.
3   """
4
5   from __future__ import annotations
6
7   import hashlib, logging, os, uuid
8   from datetime import datetime, timezone
9   from typing import Any
10
11  from fastapi import APIRouter, Depends, HTTPException, Request, status
12  from motor.motor_asyncio import AsyncIOMotorDatabase
13  from pydantic import BaseModel, EmailStr, Field
14  from passlib.context import CryptContext
15
16  from shared.redis.client import get_redis_client
17  from services.gateway.deps import (
18      create_access_token, create_refresh_token, get_current_user
19  )
20  from shared.database.session import get_db
21
22  logger = logging.getLogger(__name__)
23  router = APIRouter()
```

11.2 Pydantic schemas (lines 34–56)

```
34  class RegisterRequest(BaseModel):
35      email: EmailStr                    # validates it's a real email format
36      password: str = Field(min_length=8)  # requires 8+ chars
37      full_name: str = Field(min_length=2, max_length=100)
38      # NOTE: 'role' is intentionally NOT accepted — public registration is
always 'user'
39
40  class LoginRequest(BaseModel):
41      email: EmailStr
42      password: str
43
44  class TokenResponse(BaseModel):
45      access_token: str     # short-lived (1 hour)
46      refresh_token: str    # long-lived (7 days)
47      token_type: str = "bearer"
48      expires_in: int       # seconds
49
50  class RefreshRequest(BaseModel):
51      refresh_token: str
```

11.3 Password helpers (lines 60–75)

```
60  import bcrypt
61
62  def _hash_password(password: str) -> str:
63      """Hash password using bcrypt."""
64      pw_bytes = password.encode('utf-8')[:72]  # bcrypt max 72 bytes
65      return bcrypt.hashpw(pw_bytes, bcrypt.gensalt()).decode('utf-8')
66      # gensalt() = random salt → same password → different hash each time
67
68  def _verify_password(password: str, hashed: str) -> bool:
69      try:
70          pw_bytes = password.encode('utf-8')[:72]
71          return bcrypt.checkpw(pw_bytes, hashed.encode('utf-8'))
72          # bcrypt extracts the salt from the hash, then re-hashes and
compares
73      except Exception:
74          return False
```

11.4 Register endpoint (lines 80–131)

```
80  @router.post("/register", response_model=TokenResponse,
status_code=status.HTTP_201_CREATED)
81  async def register(
82      body: RegisterRequest,
83      db: AsyncIOMotorDatabase = Depends(get_db),
84  ) -> TokenResponse:
85      existing = await db.users.find_one({"email": body.email})
86      if existing:
87          raise HTTPException(status.HTTP_409_CONFLICT, "Email already
exists")
88
89      user_id = str(uuid.uuid4())
90      user_doc = {
91          "_id": user_id,
92          "email": body.email,
93          "full_name": body.full_name,
94          "password_hash": _hash_password(body.password),  # never store
plain!
95          "role": "user",      # always 'user' on public registration
96          "is_active": True,
97          "email_verified": False,
98          "created_at": datetime.now(timezone.utc),
99          ...
100     }
101     await db.users.insert_one(user_doc)
102
103     access_token = create_access_token(user_id, body.email, "user")
104     refresh_token = create_refresh_token(user_id)
105
106     return TokenResponse(
107         access_token=access_token,
108         refresh_token=refresh_token,
109         expires_in=3600,
110     )
```

11.5 Login endpoint (lines 134–170)

```
134 @router.post("/login", response_model=TokenResponse)
135 async def login(
136     body: LoginRequest,
137     db: AsyncIOMotorDatabase = Depends(get_db),
```

```
138 ) -> TokenResponse:
139     user = await db.users.find_one({"email": body.email})
140     if not user or not _verify_password(body.password,
user.get("password_hash", "")):
141         raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid email or
password")
142         # same error for both cases — don't reveal which one was wrong
143
144     if not user.get("is_active", True):
145         raise HTTPException(status.HTTP_403_FORBIDDEN, "Account is
deactivated")
146
147     await db.users.update_one(
148         {"_id": user["_id"]},
149         {"$set": {"last_login": datetime.now(timezone.utc)}},
150     )
151     access_token = create_access_token(str(user["_id"]), user["email"],
user.get("role", "user"))
152     refresh_token = create_refresh_token(str(user["_id"]))
153     return TokenResponse(access_token=access_token,
refresh_token=refresh_token, expires_in=3600)
```

11.6 Refresh, logout, me, change-password (lines 173–274)

```
173 @router.post("/refresh", response_model=TokenResponse)
174 async def refresh_token_endpoint(body: RefreshRequest, db=Depends(get_db)):
175     import jwt
176     secret = os.getenv("JWT_SECRET_KEY", "changeme-super-secret-
key-32chars!!")
177     algorithm = os.getenv("JWT_ALGORITHM", "HS256")
178
179     try:
180         payload = jwt.decode(body.refresh_token, secret,
algorithms=[algorithm])
181     except Exception as e:
182         raise HTTPException(status.HTTP_401_UNAUTHORIZED, f"Invalid refresh
token: {e}")
183
184     if payload.get("type") != "refresh":
185         raise HTTPException(401, "Not a refresh token")
186
187     user_id = payload.get("sub")
188     user = await db.users.find_one({"_id": user_id})
189     if not user:
190         raise HTTPException(404, "User not found")
191
192     new_access = create_access_token(user_id, user["email"],
user.get("role", "user"))
193     new_refresh = create_refresh_token(user_id)
194     return TokenResponse(access_token=new_access, refresh_token=new_refresh,
expires_in=3600)

208 @router.post("/logout")
209 async def logout(user: dict = Depends(get_current_user)):
210     """Blacklist the current JWT in Redis until it expires."""
211     jti = user.get("jti")  # JWT ID — unique per token
212     if jti:
213         exp = user.get("exp")
214         ttl = max(1, int(exp - datetime.now(timezone.utc).timestamp())) if
exp else 3600
215         redis = get_redis_client()
216         await redis.set(f"revoked_token:{jti}", "1", ex=ttl)
217     return {"message": "Logged out"}
```

```
230 @router.get("/me")
231 async def get_me(user: dict = Depends(get_current_user),
db=Depends(get_db)):
232     user_doc = await db.users.find_one({"_id": user["sub"]},
{"password_hash": 0})
233     if not user_doc: raise HTTPException(404, "User not found")
234     return {
235         "id": str(user_doc["_id"]),
236         "email": user_doc.get("email"),
237         "full_name": user_doc.get("full_name"),
238         "role": user_doc.get("role", "user"),
239         "is_active": user_doc.get("is_active", True),
240         "email_verified": user_doc.get("email_verified", False),
241         "created_at": user_doc.get("created_at").isoformat() if
user_doc.get("created_at") else None,
242         "last_login": user_doc.get("last_login").isoformat() if
user_doc.get("last_login") else None,
243     }
```

11.7 Bootstrap admin (lines 277–356)

```
288 @router.post("/create-first-admin", status_code=status.HTTP_201_CREATED)
289 async def create_first_admin(
290     body: BootstrapAdminRequest,
291     db: AsyncIOMotorDatabase = Depends(get_db),
292 ):
293     """ONE-TIME bootstrap: create the very first admin account.
294     Disabled unless BOOTSTRAP_SECRET env var is set."""
295     secret = os.getenv("BOOTSTRAP_SECRET", "")
296     if not secret:
297         raise HTTPException(404, "Bootstrap endpoint is disabled")
298     if body.bootstrap_secret != secret:
299         raise HTTPException(403, "Invalid bootstrap secret")
300
301     existing_admin = await db.users.find_one({"role": "admin"})
302     if existing_admin:
303         raise HTTPException(409, "An admin account already exists")
304     ...
305     # Create or promote to admin
```

Key takeaways

- **Pydantic validation**: Just declaring `email: EmailStr` and `password: str = Field(min_length=8)` makes FastAPI auto-validate the request body. Bad data → 422 with field-level errors, never reaches your code.
- **bcrypt over plain text**: Bcrypt is intentionally slow (configurable cost). It makes brute-force attacks expensive even if the database leaks.
- **Same error for wrong email and wrong password**: Revealing "this email exists but password is wrong" lets attackers enumerate valid emails.
- **JWT = stateless auth**: The server doesn't store sessions. The JWT contains everything (user_id, role, expiry) signed with a secret. Just verify the signature and trust the contents.
- **Refresh token in Redis for logout**: When a user logs out, we put the JWT ID (`jti`) in a Redis blacklist with a TTL matching the token's expiry. Subsequent requests with that token are rejected.

- **Bootstrap admin via shared secret**: The first admin is created by hitting `/create-first-admin` with `BOOTSTRAP_SECRET` (an env var). Once any admin exists, this endpoint refuses. This avoids the chicken-and-egg problem of "how do I get the first admin?".

Part 12: Line-by-Line: `services/gateway/routers/jobs.py`

Purpose: The main jobs CRUD API. Powers the public job board.

12.1 Background trigger helper (lines 1–41)

```
1   """Jobs CRUD router — full job management API."""
2
9   from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException,
Query, Request, status
10  from motor.motor_asyncio import AsyncIOMotorDatabase
11
12  from services.gateway.deps import get_current_user, require_admin
13  from shared.database.session import get_db
14  from shared.utils.url import normalize_url
15
20  async def trigger_search_collection(q: str, location: str | None = None) ->
None:
21      """Trigger a background Kafka collection for this search term."""
22      try:
23          from shared.kafka.producer import get_producer
24          from shared.kafka.topics import TOPICS
25          import uuid
26
27          producer = await get_producer()
28          task = {
29              "task_id": str(uuid.uuid4()),
30              "sources": ["adzuna", "greenhouse", "indeed", "rss"],
31              "search_terms": [q],
32              "location": location,
33              "limit": 100,
34              "triggered_by": "search_auto_trigger",
35          }
36          await producer.send(TOPICS.COLLECTION_TRIGGER, task)
37          logger.info(f"Triggered background collection for q='{q}'")
38      except Exception as e:
39          logger.error(...)
```

12.2 List jobs (lines 45–203) — The most important endpoint

```
45  @router.get("", response_model=dict[str, Any], summary="List jobs with
filters")
46  async def list_jobs(
47      request: Request,
48      background_tasks: BackgroundTasks,
49      q: str | None = Query(None, description="Text search"),
50      location: str | None = Query(None, description="City, state, or
country"),
51      remote: str | None = Query(None, description="remote, hybrid, on_site"),
52      job_type: str | None = Query(None, description="full_time, part_time,
contract"),
53      experience: str | None = Query(None, description="entry, mid, senior,
lead"),
```

```
    salary_min: float | None = Query(None, description="Min salary
(INR/year)"),
    salary_max: float | None = Query(None, description="Max salary
(INR/year)"),
    skills: list[str] = Query(default=[], description="Required skills"),
    company: str | None = Query(None, description="Company name filter"),
    min_trust_score: float = Query(30.0),
    max_scam_risk: str = Query("medium"),
    posted_within_days: int | None = Query(None),
    status_filter: str = Query("published"),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    sort_by: str = Query("posted_at"),
    sort_order: str = Query("desc"),
    db: AsyncIOMotorDatabase = Depends(get_db),
) -> dict[str, Any]:
    """List jobs with comprehensive filtering. Returns paginated results
with trust indicators."""

    # ── Step 1: Build MongoDB filter ──
    filters = {"status": status_filter, "is_duplicate": {"$ne": True}}
    # ↑ Show only published, non-duplicate jobs

    # ── Step 2: Text search filter ──
    if q:
        background_tasks.add_task(trigger_search_collection, q, location)
        # ↑ Fire-and-forget Kafka trigger to scrape fresh jobs
        filters["$or"] = [
            {"title": {"$regex": q, "$options": "i"}},
            {"description": {"$regex": q, "$options": "i"}},
            {"company_name": {"$regex": q, "$options": "i"}},
        ]

    if location:
        filters["$or"] = filters.get("$or", []) + [
            {"location_city": {"$regex": location, "$options": "i"}},
            ...
        ]

    if remote:        filters["remote_type"] = remote
    if job_type:      filters["job_type"] = job_type
    if experience:    filters["experience_level"] = experience
    if salary_min:    filters["salary_max"] = {"$gte": salary_min}
    if salary_max:    filters["salary_min"] = {"$lte": salary_max}

    # ── Step 3: Scam risk filter ──
    scam_risk_map = {"very_low": 0.2, "low": 0.4, "medium": 0.6, "high":
0.8, "very_high": 1.0}
    max_scam_prob = scam_risk_map.get(max_scam_risk, 0.6)
    filters["$and"] = filters.get("$and", []) + [
        {"$or": [
            {"scam_probability": {"$lte": max_scam_prob}},
            {"scam_probability": {"$exists": False}},
        ]}
    ]

    if posted_within_days:
        cutoff = datetime.now(timezone.utc) -
timedelta(days=posted_within_days)
        filters["posted_at"] = {"$gte": cutoff}

    # ── Step 4: Count + fetch page ──
    total = await db.jobs.count_documents(filters)
    sort_dir = -1 if sort_order == "desc" else 1
```

```
    cursor = db.jobs.find(filters).sort([(sort_by,
sort_dir)]).skip((page-1)*page_size).limit(page_size)
    jobs = await cursor.to_list(length=page_size)

    # ── Step 5: Fetch all companies in ONE query (no N+1) ──
    company_ids = [j.get("company_id") for j in jobs if j.get("company_id")]
    companies = []
    if company_ids:
        companies = await db.companies.find({"_id": {"$in":
company_ids}}).to_list(length=None)
    company_map = {str(c["_id"]): c for c in companies}

    # ── Step 6: Format each job for the response ──
    jobs_out = []
    for job in jobs:
        ...
        jobs_out.append({...})

    return {
        "results": jobs_out,
        "total": total,
        "page": page,
        "page_size": page_size,
        "total_pages": (total + page_size - 1) // page_size,
        "filters_applied": {...},
    }
```

12.3 Get one job (lines 206–302)

```
@router.get("/{job_id}", summary="Get job details")
async def get_job(job_id: str, db=Depends(get_db)) -> dict[str, Any]:
    job = await db.jobs.find_one({"_id": job_id})
    if not job:
        job = await db.jobs.find_one({"id": job_id})
    if not job:
        raise HTTPException(404, f"Job {job_id} not found")

    company = None
    if job.get("company_id"):
        company = await db.companies.find_one({"_id":
job.get("company_id")})

    salary_min = job.get("salary_min")
    salary_max = job.get("salary_max")
    salary_display = None
    if salary_min and salary_max:
        salary_display = f"₹{salary_min/100000:.1f}L -
₹{salary_max/100000:.1f}L/year"

    return {
        "id": str(job.get("_id")),
        "title": job.get("title"),
        "description": job.get("description"),
        "company": {...},
        "location": {...},
        "salary": {...},
        "skills": {...},
        "verification": {...},
    }
```

12.4 Other endpoints

```
305 @router.get("/{job_id}/pipeline", summary="Get job pipeline status")
306 async def get_pipeline_status(job_id: str, db=Depends(get_db)):
307     """Track a job's progress through the 8-stage pipeline."""
308     job = await db.jobs.find_one({"_id": job_id})
309     if not job: raise HTTPException(404, "Job not found")
310     events = await db.pipeline_events.find({"job_id":
str(job_id)}).sort("created_at", 1).to_list(length=None)
311     return {
312         "job_id": str(job.get("_id")),
313         "current_status": job.get("status"),
314         "events": [{"event_type": e.get("event_type"), "agent":
e.get("agent_name"),
315                     "status": e.get("status"), "duration_ms":
e.get("duration_ms"),
316                     "timestamp": e.get("created_at").isoformat() if
e.get("created_at") else None,
317                     "error": e.get("error_message")} for e in events],
318     }

337 @router.delete("/{job_id}", summary="Delete a job (admin only)")
338 async def delete_job(
339     job_id: str,
340     db: AsyncIOMotorDatabase = Depends(get_db),
341     _: Any = Depends(require_admin),     # ← admin role required
342 ):
343     result = await db.jobs.delete_one({"_id": job_id})
344     if result.deleted_count == 0:
345         result = await db.jobs.delete_one({"id": job_id})
346     if result.deleted_count == 0:
347         raise HTTPException(404, "Job not found")
348     return {"message": f"Job {job_id} deleted"}

353 @router.post("/{job_id}/report", summary="Report a suspicious job")
354 async def report_job(
355     job_id: str, reason: str, details: str | None = None,
356     db: AsyncIOMotorDatabase = Depends(get_db),
357 ):
358     """Allow users to report suspicious job postings."""
359     job = await db.jobs.find_one({"_id": job_id})
360     if not job: raise HTTPException(404, "Job not found")
361     event = {
362         "_id": str(uuid.uuid4()),
363         "job_id": str(job_id),
364         "event_type": "user.report",
365         "agent_name": "user",
366         "status": "pending_review",
367         "payload": {"reason": reason, "details": details},
368         "created_at": datetime.now(timezone.utc),
369     }
370     await db.pipeline_events.insert_one(event)
371     return {"message": "Report submitted. Our team will review this job
posting."}

383 @router.post("/collect/trigger", summary="Trigger job collection (admin)",
tags=["Admin"])
384 async def trigger_collection(
385     sources: list[str] | None = None,
386     search_terms: list[str] | None = None,
387     _: Any = Depends(require_admin),
388 ):
389     """Manually trigger a job collection run."""
390     from shared.kafka.producer import get_producer
```

```
391     from shared.kafka.topics import TOPICS
392     producer = await get_producer()
393     task = {
394         "sources": sources or ["adzuna", "greenhouse"],
395         "search_terms": search_terms or ["software engineer", "data
scientist", "python developer"],
396         "triggered_by": "admin_api",
397     }
398     await producer.send(TOPICS.COLLECTION_TRIGGER, task)
399     return {"message": "Collection trigger sent", "task": task}
```

Key takeaways

- **`background_tasks.add_task(...)`**: FastAPI runs the function AFTER returning the response, so the user doesn't wait. The function must be `async def` (or sync, FastAPI runs sync ones in a threadpool).
- **MongoDB query construction**: All filter conditions are built into a single `dict`, then passed to `db.jobs.find(filters)`. This is more efficient than running N queries with `if` checks.
- **`{"$ne": True}` matches False AND missing**: A common MongoDB trick. The deduplicator marks duplicates with `is_duplicate: True`; we want everything except that. `{"$ne": True}` matches both `False` and missing fields.
- **Avoiding N+1 queries**: Instead of fetching each job's company one at a time (N queries), we collect all `company_id`s and fetch them in ONE query using `{"$in": [...]}`. Then we build a lookup map in Python.
- **`require_admin` dependency**: `Depends(require_admin)` is a custom dependency that checks `user["role"] == "admin"`. If not, raises 403 before the endpoint runs.
- **Pagination math**: `total_pages = (total + page_size - 1) // page_size` is the standard "ceiling division" trick in Python to round up.

Part 13: The 3 ML Models

13.1 `ml/salary_estimator/estimator.py` (505 lines)

Purpose: Predict salary range (INR/year) for any job. Uses 5 lookup tables + a Gradient Boosting ML model.

Key components:

1. **5 lookup tables (rule-based brain):**

```
26  ROLE_SALARY_MAP = {
27      "software engineer":  (6.0, 12.0, 22.0),    # min, median, max in LPA
28      "python developer":   (7.0, 15.0, 28.0),
29      "data scientist":     (10.0, 18.0, 35.0),
30      "machine learning engineer": (12.0, 22.0, 45.0),
31      "engineering manager": (22.0, 35.0, 65.0),
32      ...
33  }
```

```
35  EXPERIENCE_MULTIPLIERS = {
36      "entry": 0.70, "mid": 1.00, "senior": 1.55, "lead": 2.10, "executive":
3.00,
37  }

38  LOCATION_MULTIPLIERS = {
39      "bengaluru": 1.20, "mumbai": 1.15, "delhi": 1.10, "kolkata": 0.90,
"remote": 1.05,
40  }

41  SKILL_PREMIUMS = {  # additive LPA bonus
42      "machine learning": 2.5, "llm": 4.0, "rust": 3.5, "kubernetes": 2.5, ...
43  }

44  COMPANY_MULTIPLIERS = {
45      "faang": 2.50, "mnc": 1.40, "startup": 1.10, "product": 1.30, "service":
0.85,
46  }
```

1. **Detection helpers:**

```
160 def _detect_experience_level(text: str) -> str:
161     """Keyword-based: '5+ years' → 'senior', 'principal' → 'executive'."""
162     text_lower = text.lower()
163     if any(k in text_lower for k in ["0-2", "fresher", "junior"]): return
"entry"
164     if any(k in text_lower for k in ["10+", "principal", "vp", "director"]):
return "executive"
165     if any(k in text_lower for k in ["8-", "lead", "tech lead",
"architect"]): return "lead"
166     if any(k in text_lower for k in ["5-", "senior", "sr."]): return
"senior"
167     if any(k in text_lower for k in ["2-", "mid"]): return "mid"
168     ...

189 def _detect_company_type(company_name: str) -> str:
190     if any(f in company_name.lower() for f in FAANG_COMPANIES): return
"faang"
191     if any(s in company_name.lower() for s in SERVICES_COMPANIES): return
"service"
192     return "unknown"

211 def _match_role(title: str) -> tuple:
212     """Match job title to the closest role in ROLE_SALARY_MAP."""
213     title_lower = title.lower()
214     sorted_roles = sorted(ROLE_SALARY_MAP.keys(), key=len, reverse=True)
215     for role in sorted_roles:                       # longest first
216         if re.search(r"\b" + re.escape(role) + r"\b", title_lower):
217             return ROLE_SALARY_MAP[role]
218     ...
```

1. **The main estimator class:**

```
243  class SalaryEstimator:
244      """Rule-based + optional Gradient Boosting ML fallback."""
245
246      def __init__(self, model_path: str | None = None):
247          self.model = None
248          if model_path and Path(model_path).exists():
249              try:
250                  with open(model_path, "rb") as f:
251                      bundle = pickle.load(f)
```

```
252                 self.model = bundle["model"]        # sklearn model
253                 self.feature_cols = bundle["feature_columns"]
254             except Exception:
255                 pass  # gracefully fall back to rules
256 
257     def estimate(self, title, description="", skills=None, company_name="",
258                  location=None, experience_level=None) -> dict:
259         """Returns {salary_min, salary_max, salary_median,
confidence, ...}"""
260         ...
261         # Step 1: Match role
262         role_range = _match_role(title) or (4.0, 10.0, 20.0)
263         base_min, base_med, base_max = role_range
264 
265         # Step 2: Get multipliers
266         exp_mult =
EXPERIENCE_MULTIPLIERS.get(detect_experience_level(text), 1.0)
267         loc_mult = LOCATION_MULTIPLIERS.get(detect_location(location), 1.0)
268         company_mult =
COMPANY_MULTIPLIERS.get(detect_company_type(company), 1.0)
269         skill_bump = calculate_skill_premium(skills)
270 
271         # Step 3: Compose final estimate
272         salary_min_lpa = (base_min * exp_mult * loc_mult * company_mult) +
(skill_bump * 0.5)
273         salary_max_lpa = (base_max * exp_mult * loc_mult * company_mult) +
skill_bump
274 
275         # Step 4: If ML model is available, override
276         if self.model:
277             ml_pred = self.model.predict([features])[0]
278             salary_median = ml_pred
279             salary_min = ml_pred * 0.75
280             salary_max = ml_pred * 1.35
281 
282         # Convert LPA → INR (1 LPA = 100,000 INR)
283         return {
284             "salary_min": int(salary_min_lpa * 100_000),
285             "salary_max": int(salary_max_lpa * 100_000),
286             "salary_median": int(salary_median_lpa * 100_000),
287             "currency": "INR",
288             "period": "yearly",
289             "is_estimated": True,
290             "confidence": 0.7 if ml_pred else 0.4,
291             "estimation_method": "ml_model" if self.model else
"rule_based",
292         }
```

13.2 `ml/scam_detector/model.py` (182 lines) + `train.py` (172 lines)

Purpose: Detect if a job posting is a scam. XGBoost ML + 13-rule fallback.

Feature extraction (in `train.py`):

```
38  def extract_features(job: dict) -> dict[str, float]:
39      """Convert a job dict into 22 numerical features."""
40      desc = (job.get("description") or "").lower()
41      title = (job.get("title") or "").lower()
42      company = (job.get("company_name") or "").lower()
43      apply_url = (job.get("apply_url") or "").lower()
44      source = (job.get("source") or "").lower()
45      salary_raw = (job.get("salary_raw") or "").lower()
46      skills = job.get("required_skills") or []
```

```
47
48      # Keyword features
49      n_scam_keywords = sum(1 for kw in SCAM_KEYWORDS if kw in desc)
50      desc_words = len(desc.split()) or 1
51
52      # URL features
53      has_whatsapp_url = int("whatsapp" in apply_url or "wa.me" in apply_url)
54      has_telegram_url = int("t.me" in apply_url or "telegram" in apply_url)
55      has_suspicious_domain = int(any(d in apply_url for d in
SUSPICIOUS_DOMAINS))
56      has_trusted_domain = int(any(d in apply_url for d in TRUSTED_DOMAINS))
57
58      # Source trust
59      is_trusted_source = int(source in TRUSTED_SOURCES)
60
61      # Salary features
62      has_unrealistic_salary = int(bool(re.search(r"[1-9][0-
9]\s*lakh\s*per\s*month", salary_raw)))
63      has_daily_weekly_payment = int(bool(re.search(r"\b(daily|weekly)
\s*(payment|salary|earn)", salary_raw + desc)))
64
65      # Content features
66      has_guaranteed = int("guaranteed" in desc)
67      has_registration_fee = int(bool(re.search(r"(registration|joining|
processing)\s*fee", desc)))
68      has_upfront_payment = int(bool(re.search(r"(pay|payment|deposit).{0,30}
(start|join|work)", desc)))
69      short_description = int(desc_words < 100)
70      too_many_exclamations = int(desc.count("!") > 5)
71      has_phone_number = int(bool(re.search(r"(\+91|0[6-9][0-9]{8})", desc +
title)))
72
73      # Company features
74      company_name_length = len(company.split())
75      has_pvt_ltd = int("pvt" in company or "private limited" in company)
76      suspicious_company_name = int(company_name_length <= 1 or len(company) <
4)
77
78      # Skills feature
79      num_skills = len(skills)
80
81      return {
82          "n_scam_keywords": n_scam_keywords,
83          "scam_keyword_density": n_scam_keywords / desc_words,
84          "has_whatsapp_url": has_whatsapp_url, ...
85          "description_length": desc_words,
86      }
```

Inference wrapper (in `model.py`):

```
36  class ScamDetectorModel:
37      """Inference wrapper around the trained XGBoost scam detector."""
38
39      def __init__(self, model_path=None):
40          self._model = None
41          self._loaded = False
42          self._load()  # tries to load model.pkl, falls back to rules
43
44      def _load(self):
45          path = Path(self._model_path or "ml/scam_detector/model.pkl")
46          if not path.exists(): return  # graceful — no model? use rules
47          try:
48              with open(path, "rb") as f: bundle = pickle.load(f)
```

```
            self._model = bundle["model"]
            self._loaded = True
        except Exception: pass

    def predict(self, job: dict) -> float:
        """Returns scam probability 0.0 (legit) to 1.0 (scam)."""
        features = extract_features(job)
        if self._loaded and self._model is not None:
            try:
                import numpy as np
                X = np.array([[features.get(c, 0) for c in
self._feature_cols]])
                return float(self._model.predict_proba(X)[0][1])
            except Exception:
                pass
        return self._rule_based_score(features, job)

    def classify(self, job: dict) -> str:
        prob = self.predict(job)
        if prob >= 0.7: return "scam"
        if prob >= 0.4: return "suspicious"
        return "legitimate"

    def _rule_based_score(self, features, job):
        score = 0.0
        if features.get("has_registration_fee"): score += 0.40
        if features.get("has_whatsapp_url") or
features.get("has_telegram_url"): score += 0.35
        if features.get("has_unrealistic_salary"): score += 0.25
        if features.get("has_upfront_payment"): score += 0.20
        if features.get("is_trusted_source"): score -= 0.25
        if features.get("has_trusted_domain"): score -= 0.15
        return max(0.0, min(1.0, score))
```

Training (in `train.py`):

```
def train(data_path, output_path):
    df = pd.read_csv(data_path)
    if "is_scam" not in df.columns: raise ValueError("Need 'is_scam'
column")

    features = df.apply(lambda row: extract_features(row.to_dict()), axis=1)
    X = pd.DataFrame(list(features))[FEATURE_COLUMNS]
    y = df["is_scam"].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
stratify=y)

    model = XGBClassifier(
        n_estimators=200, max_depth=5, learning_rate=0.05,
        scale_pos_weight=(y == 0).sum() / (y == 1).sum(),  # handle
imbalance
        subsample=0.8, colsample_bytree=0.8,
    )
    model.fit(X_train, y_train)
    y_prob = model.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(y_test, y_prob)
    logger.info(f"AUC: {auc:.4f}")

    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "wb") as f:
        pickle.dump({"model": model, "feature_columns": FEATURE_COLUMNS,
"auc": auc}, f)
```

13.3 `ml/skill_extractor/pipeline.py` (196 lines)

Purpose: Extract a list of skills from job description text. Uses regex (fast) + spaCy NER (richer).

Skills dictionary (master list):

```
23  SKILLS_DB = {
24      "programming_languages": ["Python", "Java", "JavaScript", "TypeScript",
"Go", "Rust", "C++", "C#", ...],
25      "web_frameworks": ["FastAPI", "Django", "Flask", "Spring Boot",
"Express.js", "React", "Vue.js", "Angular", "Next.js", ...],
26      "databases": ["PostgreSQL", "MySQL", "MongoDB", "Redis",
"Elasticsearch", "Qdrant", "Pinecone", ...],
27      "cloud_devops": ["AWS", "GCP", "Azure", "Docker", "Kubernetes",
"Terraform", "Ansible", "Jenkins", ...],
28      "data_ml": ["TensorFlow", "PyTorch", "scikit-learn", "Keras", "XGBoost",
"Spark", "Airflow", "MLflow", "LangChain", ...],
29      "messaging": ["Kafka", "RabbitMQ", "Redis Streams", "AWS SQS", "NATS",
"Pulsar"],
30      "methodologies": ["Agile", "Scrum", "Kanban", "TDD", "BDD", "CI/CD",
"DevOps", "MLOps", "SRE", "REST API", "GraphQL", ...],
31      "soft_skills": ["Problem Solving", "Team Player", "Communication",
"Leadership", "Mentoring", ...],
32  }

66  ALL_SKILLS = []
67  SKILL_CATEGORY_MAP = {}
68  for category, skills in SKILLS_DB.items():
69      for skill in skills:
70          ALL_SKILLS.append(skill)
71          SKILL_CATEGORY_MAP[skill.lower()] = category

75  _NORMALIZE_MAP = {s.lower(): s for s in ALL_SKILLS}

78  ALL_SKILLS_SORTED = sorted(ALL_SKILLS, key=len, reverse=True)  # longest
first
79  _PATTERN = "|".join(re.escape(s) for s in ALL_SKILLS_SORTED)
80  _REGEX = re.compile(r"\b(?:" + _PATTERN + r")\b", re.IGNORECASE)
```

Extraction pipeline:

```
85  class SkillExtractorPipeline:
86      def __init__(self, use_spacy=True):
87          self.nlp = None
88          if use_spacy:
89              try:
90                  import spacy
91                  self.nlp = spacy.blank("en")
92                  ruler = self.nlp.add_pipe("entity_ruler")
93                  patterns = [{"label": "SKILL", "pattern": skill} for skill
in ALL_SKILLS]
94                  ruler.add_patterns(patterns)
95              except ImportError:
96                  logger.warning("spaCy not available — using regex only")
97
98      def extract(self, text: str) -> dict:
99          if not text: return {"skills": [], "categories": {}, "confidence":
0.0}
100
101         # Step 1: Regex extraction (fast, exact)
102         matches = _REGEX.findall(text)
103         found = {}
104         for m in matches:
```

```
105             canonical = _NORMALIZE_MAP.get(m.lower(), m)
106             found[canonical] = found.get(canonical, 0) + 1
107
108         # Step 2: spaCy NER (catches variations)
109         if self.nlp and len(text) < 50_000:
110             doc = self.nlp(text)
111             for ent in doc.ents:
112                 if ent.label_ == "SKILL":
113                     canonical = _NORMALIZE_MAP.get(ent.text.lower(),
ent.text)
114                     found[canonical] = found.get(canonical, 0) + 1
115
116         # Step 3: Group by category
117         unique = list(found.keys())
118         categories = {}
119         for skill in unique:
120             cat = SKILL_CATEGORY_MAP.get(skill.lower(), "other")
121             categories.setdefault(cat, []).append(skill)
122
123         return {
124             "skills": unique,
125             "categories": categories,
126             "top_skills": [s for s, _ in sorted(found.items(), key=lambda x:
-x[1])[:10]],
127             "skill_frequency": found,
128             "confidence": min(1.0, len(unique) / 10),
129         }
```

Key takeaways

- **Rule-based + ML hybrid**: Every model in this project follows the same pattern — try ML first, fall back to hand-coded rules. The system works from day 1 with no training data, and gets better over time as `.pkl` files appear.
- **Feature engineering > algorithm choice**: Most of the heavy lifting is in `extract_features()` (turning a job dict into numbers), not in XGBoost. Good features make the ML trivial; bad features make it impossible.
- **`@lru_cache` on prediction methods**: Same job asked twice → instant cache hit. Critical when 60+ search terms × 13 sources hit the same estimator.
- **5 lookup tables > neural net for salary**: The rule-based estimator gets 70% of the way with simple multiplication. The ML model improves the last 30%.
- **`spaCy entity_ruler`**: A pattern-based NER that doesn't need training data. You give it a list of skills, it finds them in text. Fast and deterministic.
- **Regex built once, used many times**: `_REGEX = re.compile(...)` at module load. Then `_REGEX.findall(text)` is O(n) in the text length.

Part 14: All 5 Worker Services Explained

14.1 `services/deduplicator/main.py` (247 lines)

Purpose: Detect and remove duplicate jobs using SHA-256 content fingerprint.

```
25  def generate_fingerprint(title: str, company: str, location: str | None) ->
str:
26      """Normalize → hash. Same content → same hash."""
27      clean_title = title.lower().strip()
28      clean_title = re.sub(r"\b(dev|developer)\b", "developer", clean_title)
29      clean_title = re.sub(r"\b(eng|engineer)\b", "engineer", clean_title)
30      clean_title = re.sub(r"[^\w\s]", "", clean_title)
31      clean_company = re.sub(r"[^\w\s]", "", company.lower().strip())
32      normalized = f"{clean_title}|{clean_company}|{(location or '').lower()}"
33      return hashlib.sha256(normalized.encode()).hexdigest()

40  class DeduplicatorService:
41      def __init__(self):
42          self.producer = KafkaProducerClient()
43          self.consumer = KafkaConsumerClient(
44              topics=[TOPICS.JOB_CLEANED],
45              group_id="deduplicator-service",
46          )

48      async def _handle_batch(self, messages):
49          # ── Step 1: Compute fingerprints locally (no DB call) ──
50          prepared = []
51          for msg in messages:
52              fp = generate_fingerprint(
53                  msg.get("title", ""),
54                  msg.get("company_name", ""),
55                  msg.get("location_city"),
56              )
57              prepared.append({"job_id": ..., "source": msg.get("source"),
58                               "source_job_id": msg.get("source_job_id"),
59                               "fingerprint": fp, "msg": msg})
60
61          # ── Step 2: ONE DB query for all exact (source, source_job_id)
matches ──
62          source_pairs = [(p["source"], p["source_job_id"]) for p in prepared
63                          if p["source"] and p["source_job_id"]]
64          or_clauses = [{"source": s, "source_job_id": sjid} for s, sjid in
source_pairs]
65          cursor = db.jobs.find({"$or": or_clauses},
66                                {"_id": 1, "source": 1, "source_job_id": 1})
67
68          # ── Step 3: ONE DB query for all fingerprint near-duplicates ──
69          fps = [p["fingerprint"] for p in prepared]
70          cursor = db.jobs.find({"content_fingerprint": {"$in": fps}},
71                                {"_id": 1, "content_fingerprint": 1})
72
73          # ── Step 4: Classify each job ──
74          for p in prepared:
75              is_dup = False
76              # Check 1: Exact source+source_job_id match in DB
77              # Check 2: Duplicate within this batch (same source+id)
78              # Check 3: Near-duplicate by fingerprint in DB
79              # Check 4: Near-duplicate within this batch
80              ...
81
82          # ── Step 5: Bulk upsert to MongoDB ──
83          await db.jobs.bulk_write(update_ops, ordered=False)
84
85          # ── Step 6: Publish only UNIQUE jobs downstream ──
86          await self.producer.send_batch(TOPICS.JOB_DEDUPLICATED,
unique_messages)
```

The 3-query magic: Without this, you'd run 1 query per job (200 queries for 200 jobs = ~1

second). With this, you run 3 queries total for the entire batch (~15ms). 65× faster.

14.2 `services/enrichment/main.py` (323 lines)

Purpose: Add skills + salary to deduplicated jobs. Uses 3 agents in parallel.

```
30  class EnrichmentService:
31      def __init__(self):
32          self.producer = KafkaProducerClient()
33          self.consumer = KafkaConsumerClient(
34              topics=[TOPICS.JOB_DEDUPLICATED],
35              group_id="enrichment-service",
36          )
37          self.skill_agent = None
38          self.salary_agent = None
39          self.salary_estimator = None
40
41      async def start(self):
42          # Load 3 agents
43          from services.enrichment.agents.skill_extractor import
SkillExtractionAgent
44          self.skill_agent = SkillExtractionAgent()
45
46          from services.enrichment.agents.salary_extractor import
SalaryExtractionAgent
47          self.salary_agent = SalaryExtractionAgent(use_llm=True)
48
49          try:
50              from ml.salary_estimator.estimator import SalaryEstimator
51              self.salary_estimator = SalaryEstimator(model_path=...)
52          except ImportError:
53              pass
54
55          await self.producer.start()
56          await self.consumer.start()
57          await self.consumer.consume_batch(self._handle_batch,
batch_size=120)

58      async def _enrich_one(self, message):
59          """Run skill + salary extraction in PARALLEL for one job."""
60          title = message.get("title", "")
61          description = message.get("description", "")
62
63          # ── KEY OPTIMIZATION: Run both agents concurrently ──
64          skill_task = asyncio.create_task(
65              self.skill_agent.extract(title, description, use_llm=True)
66          )
67          salary_task = asyncio.create_task(
68              self.salary_agent.extract(salary_raw=...,
description=description, ...)
69          )
70          skills, salary_result = await asyncio.gather(skill_task,
salary_task)
71          # Saves 50% of time vs sequential
72
73          required = [s.normalized_name for s in skills if s.is_required]
74          nice_to_have = [s.normalized_name for s in skills if not
s.is_required]
75          tech_stack = [s.normalized_name for s in skills if s.category.value
== "technical"]
76
```

```
77          enrichment_data = {
78              "required_skills": required,
79              "nice_to_have_skills": nice_to_have,
80              "tech_stack": tech_stack,
81              "skills_data": [s.model_dump() for s in skills],
82              "experience_level": ...,
83              "salary": salary_data,
84              "status": "enriched",
85              "updated_at": datetime.utcnow(),
86          }
87          message.update(enrichment_data)
88
89          event = {
90              "_id": str(uuid.uuid4()),
91              "job_id": job_id,
92              "event_type": "job.enriched",
93              "agent_name": "enrichment",
94              "status": "success",
95              "payload": {"skills_count": len(skills), "experience_level":
exp_level},
96              "created_at": datetime.utcnow(),
97          }
98          return (job_id, message, enrichment_data, event)

99      async def _handle_batch(self, messages):
100         # Run all in parallel (semaphore-limited)
101         sem = asyncio.Semaphore(50)
102         async def _with_sem(msg):
103             async with sem:
104                 return await self._enrich_one(msg)
105         results = await asyncio.gather(*[_with_sem(m) for m in messages])
106
107         # Bulk write to MongoDB
108         bulk_ops = [UpdateOne(**u) for u in job_updates]
109         await db.jobs.bulk_write(bulk_ops, ordered=False)
110         await db.pipeline_events.bulk_write(event_inserts, ordered=False)
111
112         # Publish to next topic
113         await self.producer.send_batch(TOPICS.JOB_ENRICHED, publish_list)
```

Salary decision logic:

```
242 def _build_salary_data(salary_result, salary_estimator, ...):
243     # 1. Did LLM find explicit salary? Use it.
244     if salary_result is not None and salary_result.confidence >= 0.6:
245         return salary_result.to_dict()
246
247     # 2. Otherwise try ML model
248     if salary_estimator:
249         ml = salary_estimator.estimate(...)
250         if ml["confidence"] > salary_result.confidence:
251             return ml
252
253     # 3. Fall back to whatever LLM gave (even if low confidence)
254     return salary_result.to_dict() if salary_result else default
```

14.3 `services/verifier/main.py` (262 lines)

Purpose: Scam detection + quality scoring. Routes jobs to `verified_jobs` or `rejected_jobs`.

```
28  class VerifierService:
29      SCAM_THRESHOLD = 0.6
30
31      def __init__(self):
32          self.producer = KafkaProducerClient()
33          self.consumer =
KafkaConsumerClient(topics=[TOPICS.JOB_ENRICHED], ...)
34          self.scam_agent = None
35
36      async def start(self):
37          from services.verifier.agents.scam_detector import
ScamDetectionAgent
38          self.scam_agent = ScamDetectionAgent()
39          ...
40
41      async def _verify_one(self, message):
42          # 1. Run scam detection
43          result = await self.scam_agent.analyze(
44              job_id=..., title=..., description=..., company_name=...,
45          )
46
47          # 2. Check if apply URL is reachable
48          from shared.utils.url import verify_live_url
49          url_check = await verify_live_url(message.get("apply_url"),
timeout=2.5)
50          is_reachable = url_check.get("is_reachable", True)
51
52          # 3. Compute quality score
53          quality_score = self._compute_quality_score(message,
result.scam_probability)
54          is_rejected = result.scam_probability >= self.SCAM_THRESHOLD
55
56          # 4. Composite trust score
57          base_trust = (1.0 - result.scam_probability) * 70.0 \
58                     + (quality_score / 100.0) * 20.0
59          if is_reachable: base_trust += 10.0
60          trust_score = min(100.0, base_trust)
61
62          verification_data = {
63              "scam_probability": result.scam_probability,
64              "scam_risk_level": result.risk_level,
65              "scam_triggered_rules": result.triggered_rules,
66              "authenticity_score": 100 - result.scam_probability * 100,
67              "quality_score": quality_score,
68              "trust_score": trust_score,
69              "is_verified": result.scam_probability < 0.2 and trust_score >=
60,
70              "status": "rejected" if is_rejected else "published",
71          }
72          message.update(verification_data)
73          return (job_id, message, verification_data, event, is_rejected)

74      async def _handle_batch(self, messages):
75          ...
76          # Split into verified vs rejected
77          for result in results:
78              if result.is_rejected:
79                  rejected_list.append((result.msg, result.job_id))
80              else:
81                  verified_list.append((result.msg, result.job_id))
82
83          await db.jobs.bulk_write(update_ops, ordered=False)
84
85          # Publish to TWO topics
```

```
86          if verified_list:
87              await self.producer.send_batch(TOPICS.JOB_VERIFIED,
verified_list)
88          if rejected_list:
89              await self.producer.send_batch(TOPICS.JOB_REJECTED,
rejected_list)

90      def _compute_quality_score(self, job, scam_prob) -> float:
91          score = 50.0   # neutral start
92          desc = job.get("description", "")
93          if len(desc) > 500:    score += 15
94          elif len(desc) > 200:  score += 10
95          elif len(desc) > 50:   score += 5
96          skills = job.get("required_skills", [])
97          if len(skills) >= 5:   score += 15
98          elif len(skills) >= 2: score += 10
99          if job.get("salary_min") and job.get("salary_max"): score += 10
100         if job.get("apply_url"): score += 5
101         score -= scam_prob * 40  # big penalty for scams
102         return max(0.0, min(100.0, score))
```

Trust score formula:

```
trust = (1 − scam_prob) × 70  +  (quality / 100) × 20  +  (10 if URL reachable)
       70% weight              20% weight            10% weight
```

14.4 `services/embedder/main.py` (231 lines)

Purpose: Convert verified job text to 384-dim vectors, store in Qdrant + MongoDB.

```
34  class EmbedderService:
35      def __init__(self):
36          self.consumer = KafkaConsumerClient(
37              topics=[TOPICS.JOB_VERIFIED],
38              group_id="embedder-service",
39          )
40          self.model = None       # SentenceTransformer
41          self.qdrant_client = None
42
43      async def start(self):
44          try:
45              from sentence_transformers import SentenceTransformer
46              self.model = SentenceTransformer("all-MiniLM-L6-v2")
47          except ImportError:
48              logger.warning("sentence-transformers not installed — skipping
vectorization")
49
50          try:
51              from qdrant_client import QdrantClient
52              self.qdrant_client = QdrantClient(host="qdrant", port=6333)
53              if "job_embeddings" not in [c.name for c in
self.qdrant_client.get_collections().collections]:
54                  self.qdrant_client.create_collection(
55                      collection_name="job_embeddings",
56                      vectors_config=VectorParams(size=384,
distance=Distance.COSINE),
57                  )
58          except Exception:
59              pass  # graceful — works without Qdrant
60
61          await self.consumer.start()
```

```
62          await self.consumer.consume_batch(self._handle_batch, batch_size=96)

63      async def _handle_batch(self, messages):
64          # Build text inputs
65          embed_texts = []
66          for m in messages:
67              text = f"{m['title']}. {m['company_name']}. {m['location']}.
Skills: {...}. {m['description'][:1000]}"
68              embed_texts.append(text)
69
70          # Encode ALL texts in one call (batch inference is 10-100x faster)
71          loop = asyncio.get_event_loop()
72          vectors_np = await loop.run_in_executor(
73              None,
74              lambda: self.model.encode(embed_texts, batch_size=64),
75          )
76          # run_in_executor because encode() is CPU-heavy; don't block event
loop
77
78          # Upsert to Qdrant in one call
79          points = [PointStruct(id=uuid4(), vector=v.tolist(), payload=p)
80                    for v, p in zip(vectors_np, payloads)]
81          self.qdrant_client.upsert("job_embeddings", points=points)
82
83          # Bulk update MongoDB
84          bulk_ops = [UpdateOne({"_id": jid}, {"$set": {
85              "embedding_id": eid, "embedding_model": "all-MiniLM-L6-v2",
86              "status": "published",
87          }}, upsert=True) for jid, eid in zip(job_ids, embedding_ids)]
88          await db.jobs.bulk_write(bulk_ops, ordered=False)
```

14.5 `services/notifier/main.py` (206 lines)

Purpose: Match verified jobs against user SavedSearches, send notifications.

```
25  class NotifierService:
26      def __init__(self):
27          self.consumer = KafkaConsumerClient(
28              topics=[TOPICS.JOB_VERIFIED],
29              group_id="notifier-service",
30          )
31
32      async def start(self):
33          await self.consumer.start()
34          await self.consumer.consume(self._handle_message)  # single-message
mode
35
36      async def _handle_message(self, message):
37          job_id = message.get("_id")
38          title = message.get("title", "")
39          company = message.get("company_name", "")
40
41          # Find all saved searches with alerts enabled
42          saved_searches = await db.saved_searches.find({"alert_enabled":
True}).to_list(length=500)
43
44          matched_users = []
45          for search in saved_searches:
46              if self._job_matches_search(message, search):
47                  matched_users.append({...})
48
```

```
49          if not matched_users: return
50
51          for match in matched_users:
52              user = await db.users.find_one({"_id": match["user_id"]})
53              channels = self._get_user_channels(user)   # in_app, email,
telegram, webhook
54              for channel in channels:
55                  await db.notification_logs.insert_one({
56                      "user_id": match["user_id"],
57                      "channel": channel,
58                      "subject": f"New match: {title} at {company}",
59                      "status": "sent",
60                      ...
61                  })
62                  await self._send_notification(channel, user, message, match)
63                  # ↑ Currently simulated — would call SendGrid, Telegram API,
webhook URL

64      def _job_matches_search(self, job, search) -> bool:
65          query = search.get("query", "").lower()
66          filters = search.get("filters", {})
67          if query and not any(query in t for t in [
68              job.get("title", "").lower(),
69              job.get("description", "").lower(),
70              job.get("company_name", "").lower(),
71          ]): return False
72          if filters.get("location") and filters["location"] not in
job.get("location_city", "").lower():
73              return False
74          if filters.get("remote_type") and filters["remote_type"] !=
job.get("remote_type"):
75              return False
76          if filters.get("experience_level") and filters["experience_level"] !
= job.get("experience_level"):
77              return False
78          if filters.get("salary_min") and (job.get("salary_max") or 0) <
filters["salary_min"]:
79              return False
80          return True
```

Part 15: The Universal Worker Pattern

Every worker service in this project follows the **exact same 4-phase structure**:

```
┌─────────────────────────────────────────────────────────────┐
│ PHASE 1: SETUP                                              │
│   __init__  → create consumer, producer, agents             │
│   start()   → load models, start clients, begin consuming   │
├─────────────────────────────────────────────────────────────┤
│ PHASE 2: PROCESS ONE (worker-specific)                      │
│   _process_one(msg) → run ML/LLM/rules, return enriched msg  │
├─────────────────────────────────────────────────────────────┤
│ PHASE 3: BATCH HANDLER (universal)                          │
│   _handle_batch(msgs) →                                     │
│     1. Process all in parallel (asyncio.Semaphore)          │
│     2. Bulk write to MongoDB (UpdateOne, InsertOne)         │
│     3. Bulk insert events (pipeline_events)                 │
│     4. send_batch to next Kafka topic                       │
│     5. Log performance metrics (j/s)                        │
├─────────────────────────────────────────────────────────────┤
│ PHASE 4: ENTRYPOINT                                         │
```

```
main() → instantiate service, call start()
```

The template in one snippet

```
class MyWorkerService:
    def __init__(self):
        self.producer = KafkaProducerClient()
        self.consumer = KafkaConsumerClient(
            topics=[TOPICS.MY_INPUT],
            group_id="my-worker-service",
        )
        self.running = False
        self._agent = None

    async def start(self):
        self._agent = SomeAgent()    # load models/agents here
        await self.producer.start()
        await self.consumer.start()
        self.running = True
        await self.consumer.consume_batch(self._handle_batch, batch_size=100,
timeout_ms=2000)

    async def _process_one(self, message) -> dict | None:
        # Worker-specific logic (run ML model, extract skills, etc.)
        result = self._agent.do_something(message)
        return (message["_id"], message, result_update, event_doc)

    async def _handle_batch(self, messages):
        # 1. Process all in parallel (capped at 50)
        sem = asyncio.Semaphore(50)
        async def _with_sem(msg):
            async with sem:
                return await self._process_one(msg)
        results = await asyncio.gather(*[_with_sem(m) for m in messages])

        # 2. Build bulk operations
        job_updates = [{"filter": {"_id": r[0]}, "update": {"$set": r[2]},
"upsert": True} for r in results if r]
        event_inserts = [r[3] for r in results if r]
        publish_list = [(r[1], r[0]) for r in results if r]

        # 3. Bulk write
        try:
            await db.jobs.bulk_write([UpdateOne(**u) for u in job_updates],
ordered=False)
        except Exception:
            # Fallback to per-job update
            for jid, _, data, _ in results:
                try: await db.jobs.update_one({"_id": jid}, {"$set": data},
upsert=True)
                except: pass
        await db.pipeline_events.bulk_write([InsertOne(e) for e in
event_inserts], ordered=False)

        # 4. Publish to next topic
        if publish_list:
            await self.producer.send_batch(TOPICS.MY_OUTPUT, publish_list)

        # 5. Log
        logger.info(f"Processed {len(results)}/{len(messages)} jobs in
{elapsed_ms}ms")
```

```
async def main():
    service = MyWorkerService()
    await service.start()

if __name__ == "__main__":
    asyncio.run(main())
```

Why this pattern?

- **Idempotent**: Re-running a batch is safe (MongoDB upserts, Kafka offsets track what we've already processed).
- **Parallel by default**: `asyncio.gather` + `Semaphore` means each batch uses all CPU cores but never overwhelms downstream services.
- **Bulk everywhere**: `bulk_write`, `send_batch`, single `INSERT...VALUES (...), (...), (...)`. 10-100× faster than per-record operations.
- **Graceful degradation**: Every try/except has a fallback. DB write fails? Update one-by-one. Model not found? Use rules. Kafka send fails? Log it and move on (the DB still got the update).
- **Observable**: Every batch logs `processed/total in Xms (Y j/s)`. Combined with Prometheus, you get a real-time dashboard of pipeline throughput.

Performance numbers (approximate, single instance)

Worker	Batch	Concurrency	Throughput
Deduplicator	200	40	1,000+ jobs/sec
Enrichment	120	50	50–100 jobs/sec (LLM-bound)
Verifier	180	70	200+ jobs/sec (ML is fast)
Embedder	96	40	100 jobs/sec (CPU-bound)
Notifier	N/A	N/A	Varies by user count

Part 16: Final Cheat Sheet

Architecture one-liner

```
Web → Collector → Cleaner → Deduplicator → Enrichment → Verifier → Embedder →
Notifier
                              ↓                ↓              ↓         ↓
↓
                           MongoDB          MongoDB          MongoDB  Qdrant+
MongoDB
                                                                          MongoDB
                              ↑                                             ↑
                              └──────────────  API Gateway (REST)  ──────────┘
```

```
                                ↓
                Job Board (:3000) + Admin (:3001)
```

The 5 lookup tables (salary estimator brain)

Table	Example
`ROLE_SALARY_MAP`	`"python developer"` → `(7, 15, 28)` LPA
`EXPERIENCE_MULTIPLIERS`	`"senior"` → `1.55×`, `"entry"` → `0.70×`
`LOCATION_MULTIPLIERS`	`"bengaluru"` → `1.20×`, `"kolkata"` → `0.90×`
`SKILL_PREMIUMS`	`"llm"` → `+4 LPA`, `"kubernetes"` → `+2.5 LPA`
`COMPANY_MULTIPLIERS`	`"faang"` → `2.50×`, `"service"` → `0.85×`

Formula: `salary = base × exp × loc × company + skill_bump`

The 22 scam features

Feature	Meaning
`n_scam_keywords`	Count of scam words in description
`has_whatsapp_url` / `has_telegram_url`	Apply link is a chat app
`has_suspicious_domain`	bit.ly, t.me, etc.
`has_trusted_domain`	google.com, microsoft.com, etc.
`is_trusted_source`	From Greenhouse, Lever, etc.
`has_unrealistic_salary`	"50 lakh per month"
`has_registration_fee`	Asks for money
`short_description`	< 100 words
`suspicious_company_name`	1 word or < 4 chars

3-tier classification

```
prob = self.predict(job)
if prob >= 0.7: return "scam"
if prob >= 0.4: return "suspicious"
return "legitimate"
```

Service-to-port mapping

Service	Port	Type
Frontend Job Board	3000	React UI

Service	Port	Type
Frontend Admin	3001	React UI
API Gateway	8000	FastAPI
Redpanda Console	8080	Kafka UI
Grafana	3002	Dashboards
Prometheus	9090	Metrics
Qdrant	6333	Vector DB UI
MLflow	5000	ML tracking
MinIO	9001	Object storage UI
Elasticsearch	9200	Search
MongoDB	27017	Database
Redis	6379	Cache
Redpanda	9092	Kafka

Job status flow

```
raw → cleaned → deduplicated → enriched → verified → published
                    ↘ duplicate     ↘ rejected
```

The graceful degradation pattern (recurring everywhere)

```
try:
    ml_result = model.predict(features)        # try ML first
except Exception:
    ml_result = None
if ml_result is None:
    return rule_based_fallback(features)       # always works
```

What each pattern solves

Pattern	Solves
Singleton DB client	Connection explosion
Circuit Breaker	Hammering dead sites
Adaptive rate limit	Getting 429-banned
Exponential backoff	Network blips
Bulk write	Slow per-record operations
`upsert=True`	Idempotent reprocessing
Manual offset commit	At-least-once delivery
Fingerprinting	Near-duplicate detection
Graceful degradation	Missing dependencies
`asyncio.gather` + `Semaphore`	CPU underutilization
`Depends(get_current_user)`	Boilerplate auth

Pattern	Solves
Pydantic models	Manual validation

Part 17: Useful Commands

Run the platform

```
# Copy env template and edit
cp .env.example .env
# Add: ADZUNA_API_ID, ADZUNA_API_KEY (free at developer.adzuna.com)

# Start everything (19 containers)
docker compose up -d

# View logs
docker compose logs -f gateway
docker compose logs -f enrichment

# Restart one service after code change
docker compose up -d --build enrichment

# Stop everything
docker compose down              # keeps volumes (data)
docker compose down -v           # wipes all data
```

Train ML models

```
# Salary estimator
python ml/salary_estimator/estimator.py \
    --train --data data/labeled_salaries.csv

# Scam detector
python ml/scam_detector/train.py --data data/labeled_jobs.csv

# After training, the .pkl file appears in ml/<model>/ and is auto-loaded
```

Quick predictions (CLI)

```
# Estimate a salary
python ml/salary_estimator/estimator.py \
    --title "Senior Python Developer" \
    --skills "Python,FastAPI,AWS,LLM" \
    --location "Bangalore" \
    --company "Google"

# Extract skills from text
python ml/skill_extractor/pipeline.py \
    --text "We need 5 years Python, FastAPI, AWS, and Kubernetes"
```

Database access

```
# MongoDB shell
docker exec -it jip-mongodb mongosh -u admin -p admin123

# In mongosh:
```

```
> use jobplatform
> db.jobs.find().limit(2).pretty()
> db.jobs.countDocuments({"status": "published"})
> db.pipeline_events.find({"job_id": "abc-123"}).sort({created_at: 1})
```

Kafka topics

```
# List topics
docker exec -it jip-redpanda rpk topic list --brokers localhost:9092

# Watch a topic
docker exec -it jip-redpanda rpk topic consume raw_jobs --brokers localhost:9092

# Topic stats
docker exec -it jip-redpanda rpk topic describe raw_jobs --brokers
localhost:9092
```

Tests

```
# Unit tests (no infrastructure needed)
pytest tests/unit/ -v

# Integration tests (needs running containers)
pytest tests/integration/ -v

# With coverage
pytest --cov=services --cov=shared --cov-report=html
```

Debugging

```
# Check service health
curl http://localhost:8000/health

# Trigger a manual collection
curl -X POST http://localhost:8000/api/v1/jobs/collect/trigger \
    -H "Authorization: Bearer <admin_token>" \
    -H "Content-Type: application/json" \
    -d '{"sources": ["adzuna"], "search_terms": ["python"]}'

# View Qdrant collections
curl http://localhost:6333/collections

# View MongoDB indexes
docker exec -it jip-mongodb mongosh -u admin -p admin123 --eval
"db.jobs.getIndexes()"
```

Part 18: Glossary of Key Concepts

Architecture

Term	Meaning
Microservice	A small, independent service with one job. Talks to others via API or message queue.
Event-driven	Services communicate by sending/receiving events (messages), not by calling each other

Term	Meaning
	directly.
Kafka topic	A named stream of messages. Producers write to it, consumers read from it.
Consumer group	A group of consumers that share the work for a topic. Kafka load-balances partitions among them.
At-least-once delivery	Guarantee that a message is delivered at least once. We may see duplicates, but never lose messages. (We use this via manual offset commit.)
Idempotent	Doing the same operation twice has the same effect as doing it once. (e.g., MongoDB upsert is idempotent.)
DLQ (Dead Letter Queue)	A separate topic where failed messages go. `raw_jobs.dlq`, `cleaned_jobs.dlq`, etc.
Container	A lightweight, isolated environment. Same code runs the same way on dev, staging, prod.

Backend

Term	Meaning
FastAPI	A modern Python web framework. Auto-generates OpenAPI docs, validates request bodies with Pydantic.
Pydantic	A library for data validation using Python type hints. `class Job(BaseModel)` → FastAPI auto-validates request bodies.
Motor	Async MongoDB driver for Python. `await collection.find_one(...)` instead of blocking calls.
Pymongo	Sync MongoDB driver. We use it inside the `bulk_write` helpers.
bcrypt	Password hashing function. Intentionally slow to make brute force expensive.
JWT (JSON Web Token)	A signed token containing user info. Server doesn't store sessions — the token IS the session.
HMAC	Symmetric signing used by JWT (HS256). Same secret signs and verifies.
OAuth2 password flow	The standard "give me username + password, get back a token" flow used by `/login`.
CORS	Cross-Origin Resource Sharing. Browser

Term	Meaning
	security feature. `localhost:3000` can't call `localhost:8000` unless the server says so.
`Depends(get_db)`	FastAPI's dependency injection. The function runs before the endpoint; result is passed in.
`BackgroundTasks`	FastAPI lets you schedule async work AFTER returning the response.

Async Python

Term	Meaning
`async def`	Defines a coroutine. Must be `await`ed to run.
`await`	Wait for an async operation without blocking the event loop.
`asyncio.gather(*tasks)`	Run many coroutines in parallel; wait for ALL to finish.
`asyncio.create_task(coro)`	Start a coroutine in the background, return a handle.
`asyncio.Semaphore(N)`	Allow at most N concurrent holders. Use to cap parallelism.
`asyncio.Lock()`	Mutual exclusion. Only one coroutine can hold it at a time.
`run_in_executor(func)`	Run a sync/CPU-heavy function in a thread, don't block the event loop.

Kafka

Term	Meaning
Producer	Something that writes messages to a topic.
Consumer	Something that reads messages from a topic.
Offset	A position in a topic. "I've processed up to message 1234." Committed offsets track progress.
Partition	A topic is split into partitions for parallelism. Each partition is ordered; across partitions, no order.
Acknowledgment (`acks`)	Producer waits for `acks=all` → all replicas wrote the message before considering it sent.
Idempotence	Even if the producer retries, the broker won't write a duplicate.
LZ4 compression	Fast compression algorithm. Used here to shrink JSON job payloads.

MongoDB

Term	Meaning
Document	A JSON-like record. Stored in a collection.
Collection	A group of documents. Like a table in SQL.
Index	A data structure that makes queries fast. Without it, MongoDB scans every document.
`_id`	The unique identifier of a document. By default MongoDB generates one; we use UUIDs.
`$in`	Match if field is any of the listed values. Used for batch lookups.
`$or` / **`$and`**	Logical combinators.
`$ne`	"Not equal to". Common trick: `{"is_duplicate": {"$ne": True}}` matches False and missing.
`$regex`	Pattern match. `{"$options": "i"}` makes it case-insensitive.
`upsert`	Insert if not exists, update if exists. Idempotent.
`bulk_write([op1, op2, ...])`	Execute many operations in one round-trip. 10-100× faster.
`ordered=False`	In a bulk write, don't fail the whole batch if one op fails.

Data Pipeline

Term	Meaning
ETL	Extract, Transform, Load. The three stages of any data pipeline.
Streaming vs batch	Streaming = process one event at a time as it arrives. Batch = collect N, process all at once. We use batch for throughput.
Throughput	Jobs processed per second.
Latency	Time from "job enters pipeline" to "job is published".
Backpressure	When downstream is slower than upstream. Kafka absorbs this.
DLQ	Messages that fail processing go to a Dead Letter Queue for inspection.

Web Scraping

Term	Meaning
User-Agent	A header that identifies the client. Rotating UAs avoids detection.
Rate limiting	The website's "max N requests per minute" rule. Hitting it → 429.
429 / 503	HTTP "Too Many Requests" / "Service Unavailable". The site is angry.
Circuit breaker	A pattern: after N failures, stop trying for a while. Like an electrical fuse.
Exponential backoff	Wait 1s, then 2s, then 4s, then 8s before retrying.
Jitter	Random variation in the wait time, so all parallel scrapers don't retry at the same instant.
`async with c as collector`	Python's async context manager. Opens HTTP session on enter, closes on exit.
`httpx.AsyncClient`	The async HTTP client we use. Like `requests` but non-blocking.

Machine Learning

Term	Meaning
Feature	A number extracted from raw input. ML models consume features, not text.
XGBoost	Gradient boosted decision trees. Strong baseline for tabular classification.
Gradient Boosting (sklearn)	Same idea as XGBoost, slightly different implementation.
Regex	Pattern matching on text. Fast, deterministic, no training.
NER (Named Entity Recognition)	NLP technique for finding "entities" (people, places, skills) in text.
`spaCy entity_ruler`	A pattern-based NER. Give it a list of words, it finds them in text. No training needed.
LPA	Lakhs Per Annum. Indian salary unit. 1 LPA = ₹100,000 per year.
`pickle.dump` / `pickle.load`	Serialize a Python object (like a trained model) to disk and back.
`@lru_cache`	Memoization. Caches the result of a function based on its arguments.
`predict_proba`	For classification, returns the probability of each class (not just the predicted class).

Term	Meaning
`fit` / `predict`	The two main sklearn methods. `fit(X, y)` trains; `predict(X)` infers.
`train_test_split`	Split data into 80% train, 20% test. Never test on training data.
`scale_pos_weight`	XGBoost parameter. Tells the model "scams are rare, weight them more".
Graceful degradation	If ML is missing/broken, use hand-coded rules. Always works, less accurate.
AUC-ROC	Area Under the ROC Curve. 1.0 = perfect, 0.5 = random guessing. Standard for binary classification.

Observability

Term	Meaning
Prometheus	A time-series metrics database. Services expose `/metrics`, Prometheus scrapes it.
Grafana	A dashboard tool. Connects to Prometheus and draws graphs.
Structured logging	Logs as JSON, not free text. Makes them searchable and parseable.
Health check	A `/health` endpoint that returns 200 if the service is alive, 503 otherwise. Docker and Kubernetes poll this.
MLflow	An experiment tracking tool. Records parameters, metrics, and artifacts for each training run.

Where to learn more

Topic	Resource
FastAPI	https://fastapi.tiangolo.com/tutorial/
Async Python	https://docs.python.org/3/library/asyncio.html
Kafka	https://kafka.apache.org/quickstart
MongoDB	https://docs.mongodb.com/manual/tutorial/
Docker	https://docs.docker.com/get-started/
Microservices patterns	https://microservices.io/patterns/index.html
ML deployment	https://ml-ops.org/

This walkthrough covers approximately 70% of the project's Python code. The remaining 30% is split among the frontend (React/Vite), the analytics/feedback/admin routers, the Kubernetes manifests, and various utility scripts — all built on the patterns described above.

Document generated: 2026-09-02. Source: complete codebase of TALENTLENS job intelligence platform.